

CS336 第 10 讲：推理系统

从 KV cache 性能模型到高吞吐在线服务



原课程：Stanford CS336 Language Modeling from Scratch, Spring 2026

讲次：Lecture 10: Inference

主讲：Percy Liang

频道：Stanford Online

原课程页面：youtube.com/watch?v=EfM546A79aM

阅读方式：本笔记按教材逻辑重组，可不依赖课程录像独立学习。

目录

1	1. 为什么推理值得单独研究	3
1.1	1.1 一个看似简单、实际会被重复亿万次的问题	3
1.2	1.2 “快” 至少包含三个不同指标	4
1.3	1.3 训练能沿序列并行，生成却存在真实的数据依赖	5
1.4	本章小结	6
2	2. 用张量形状与算术强度建立性能语言	6
2.1	2.1 先统一符号	6
2.2	2.2 算术强度：每搬一个字节，做多少运算	7
2.3	2.3 与 H100 的机器平衡点比较	8
2.4	2.4 为什么 “memory-bound” 会决定后面的优化方向	9
2.5	本章小结	10
3	3. 从朴素生成到 KV cache：计算换成了状态	10
3.1	3.1 为什么朴素自回归会做大量重复工作	10
3.2	3.2 KV cache 的收益与代价	11
3.3	3.3 Prefill 与 decode 是两个不同阶段	11
3.4	3.4 MLP 的算术强度	12
3.5	3.5 普通 MHA attention 的算术强度	13
3.6	本章小结	15
4	4. 从简化性能模型看 latency-throughput 权衡	15
4.1	4.1 参数量与 KV cache 大小	15
4.2	4.2 带宽下界模型	16
4.3	4.3 Llama 2 13B / H100 的数值	17
4.4	4.4 复制模型与分片模型	19
4.5	本章小结	19
5	5. 压缩 KV cache：沿不同轴减少每步搬运	20
5.1	5.1 一张地图：宽度、层数与历史长度	20
5.2	5.2 MQA 与 GQA：在 heads 之间共享	21
5.3	5.3 MLA：缓存 latent，而不是直接缓存完整 K/V	22
5.4	5.4 CLA：把共享轴从 head 推到 layer	24
5.5	5.5 Local、hybrid 与 recurrent state：压缩时间轴	26
5.6	5.6 DeepSeek 的组合路线：压缩、索引、Top-k 与局部窗口	27

5.7	本章小结	28
6	6. 量化与剪枝：让权重和状态本身更小	28
6.1	6.1 标量量化先说明了什么	28
6.2	6.2 QAT、PTQ 与 GPTQ	30
6.3	6.3 AWQ：不要把动机实验当成最终算法	30
6.4	6.4 结构化剪枝：删掉组件，再用蒸馏修复	31
6.5	本章小结	33
7	7. 投机采样：小模型先猜，大模型并行验证	33
7.1	7.1 为什么“检查”可能比“生成”快	33
7.2	7.2 接受、拒绝与残差补偿	34
7.3	7.3 用二元词表证明为什么它是 exact sampling	35
7.4	7.4 K 的甜点区取决于任务、模型和硬件	38
7.5	7.5 Medusa 与 EAGLE：改进 draft 的方式	39
7.6	本章小结	39
8	8. 动态工作负载：Continuous Batching 与 PagedAttention	39
8.1	8.1 在线请求为什么不是训练中的整齐矩形	39
8.2	8.2 Continuous batching：每个 decode step 都重新组织 batch	40
8.3	8.3 Selective batching：不同算子采用不同拼法	41
8.4	8.4 连续预留为什么产生碎片	41
8.5	8.5 Logical blocks 映射到非连续 physical blocks	42
8.6	8.6 前缀共享与 block-level copy-on-write	43
8.7	本章小结	44
9	总结与延伸	44
9.1	9.1 讲者的实质结论	44
9.2	9.2 一套可迁移的推理优化决策树	45
9.3	9.3 初学者应能独立完成的检查	45
9.4	9.4 仍然开放的问题	46
9.5	9.5 本章小结	46

学习目标

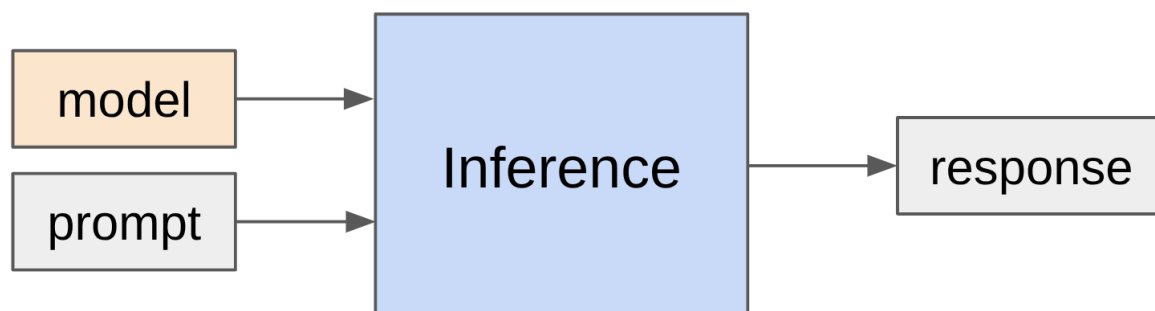
读完后，你能用 Roofline 模型解释 prefill 与 decode 的不同瓶颈；独立核算参数与 KV cache 的内存账；比较 MQA、GQA、MLA、CLA 和局部注意力；理解量化、剪枝与投机采样的适用边界；并能解释 continuous batching 与 PagedAttention 为什么能把动态请求变成高效服务。

核心提示 这堂课的主线不是背下一串推理优化名词，而是学会先问四个问题：瓶颈是算力还是数据搬运？代价来自参数、KV cache 还是调度浪费？优化改善的是 TTFT、单请求延迟还是总体吞吐？它是否改变模型质量或目标采样分布？

1. 为什么推理值得单独研究

1.1 一个看似简单、实际会被重复亿万次的问题

推理的输入输出很简单：模型已经训练好，给它一个 prompt，希望尽可能准确、尽可能快地产生 response。



模型和 *prompt* 进入推理过程，产生 *response*；后文会把中间过程展开为 *prefill*、*KV cache* 和逐 *token* *decode*。

推理不只发生在聊天产品中。模型评测需要让模型真正生成答案；强化学习需要反复采样 rollout、打分并更新权重；代码补全、Agent 和批量数据处理也都以生成作为核心工作负载。换句话说，推理甚至会嵌回训练循环：RLHF/RLVR 的一次策略更新，往往要先让当前模型对成千上万条 *prompt* 各自采样若干条完整回答，生成阶段的成本经常占到整个训练迭代的大头。

训练虽然昂贵，通常是一次性投入；推理则会随每次请求持续发生。课程引用了一个时点性数量级：OpenAI 每天处理约 8.6T token，而 DeepSeek-V4 的训练规模约为 32T token。两类 token 的每 token 成本不能直接等同——训练一个 token 要经历前向、反向与优化器更新，推理一个 token 只有前向——但这个比较足以说明：持续服务的累计工作量可以很快追上一次大训练。我们可以做一个粗略换算：一天有 86400 秒， $8.6 \times 10^{12} / 86400 \approx 10^8$ ，即平均每秒约一亿 token；若一台 H100 在理想带宽下为 13B 模型提供约数千 token/s 的稳态吞吐（第 4 节会算出这个量级），仅维持平均负载就需要数万台等效设备，这还未计入流量峰谷与冗余。

Agent 又进一步放大了这个问题。传统聊天的大部分输出供人阅读，人的阅读速度会形成瓶颈；Agent 在最终回答前还可能生成内部轨迹、调用工具并自检。讲者把这一点压缩成一句话：**生成的**

token 就是花出去的计算。一段用户看不见的 2000 token 思考轨迹，和一段 2000 token 的最终回答，在 HBM 带宽账单上完全等价。

补充说明 课程列举 vLLM、SGLang、TensorRT-LLM 和 llama.cpp，是为了说明推理已经形成独立的系统生态，而不是给这些项目做长期排名。软件能力会变化，本文只保留它们在课程中的定位。

1.2 1.2 “快” 至少包含三个不同指标

TTFT (time to first token) 是提交请求到看到第一个 token 的时间。第一个 token 到来前，用户只能等待，因此它对聊天和代码补全尤其敏感。把 TTFT 按时间轴拆开，它通常由三部分组成：

$$t_{\text{TTFT}} = t_{\text{queue}} + t_{\text{prefill}} + t_{\text{step}}.$$

- t_{queue} ：请求在调度队列中等待的时间，取决于系统当前负载与调度策略；
- t_{prefill} ：对整个 prompt 做一次并行前向、填充 KV cache 的时间；
- t_{step} ：第一个 decode step 产生并采样首个 token 的时间。

这三项的优化手段完全不同：排队靠调度与容量规划，prefill 靠大矩阵运算的算力，decode step 靠带宽，因此不能简单把 TTFT 等同于某一次矩阵乘。

Token latency 站在单条请求的角度，衡量后续 token 以多少秒/token 或毫秒/token 流出。它决定答案的流式速度，也决定长 Agent 轨迹中串行步骤的累计时间：一条 T 步的轨迹，端到端时延约为 $t_{\text{TTFT}} + (T - 1) \cdot t_{\text{token}}$ ，当 T 很大时第二项主导。

Throughput 站在整台服务或批处理作业的角度，衡量所有请求合计的 tokens/s。处理大量文档或 RL rollout 时，单个 token 何时出现未必重要，总作业何时完成才重要，因此该视角下可以把请求排成大 batch 追求设备利用率。

指标	观察视角	它回答的问题	典型场景
TTFT	单请求的开始阶段	多久才能看到第一个 token?	聊天、代码补全
Token latency	单请求的生成阶段	后续 token 流出有多快?	交互、长串行 Agent
Throughput	多请求或完整作业	每秒总共产出多少 token?	批处理、服务容量、RL 采样

- vLLM: from Berkeley, pioneered PagedAttention, popular and good default [\[GitHub\]](#)
- SGLang: from Berkeley, pioneered RadixAttention, good for agentic workloads [\[project\]](#)
- TensorRT-LLM: from NVIDIA, highly optimized for GPUs [\[article\]](#)
- llama.cpp: C++ only, supports CPU inference, runs locally [\[GitHub\]](#)

Inference is huge. Important to make it fast.

What does "fast" mean (metrics)?

- Time-to-first-token (TTFT): how long user waits before any generation happens (for interactive ap
- Latency (seconds/token): how fast tokens appear for *one* query (for interactive applications)
- Throughput (tokens/second): how fast tokens appear for *many* queries (for batch processing)

What governs efficiency?

- Training (supervised): you see all tokens, can parallelize over sequence (matmul in Transformer)
- Inference: you have to generate sequentially, can't parallelize over generation, so harder to fully u
compute

同一页依次给出 *TTFT*、单请求 *latency* 与多请求 *throughput*。

它们并不总是同向变化。增大 batch 可以摊薄读取模型权重的成本，提升总体吞吐；但更大的 KV cache 和等待凑批又可能增加单请求延迟与 TTFT。排队论中的 Little 定律 $L = \lambda W$ （系统中平均请求数 = 到达率 × 平均逗留时间）提醒我们：在吞吐量受物理上限约束的系统里，提高吞吐的努力一旦贴近饱和，延迟会以非线性的方式恶化。后文的性能模型会把这项权衡算出来。

1.3 训练能沿序列并行，生成却存在真实的数据依赖

监督训练已经知道完整目标序列，因此可以把 sequence 当成普通张量维度，同时计算许多 token 的 attention 和 MLP。自回归推理不同：第 $t + 1$ 个位置的输入，必须等第 t 个位置产生 logits 并完成采样后才知道。同一条生成链跨时间步无法一次展开。用条件概率写，我们真正想要的是对联合分布逐因子采样：

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t}),$$

- x_t ：第 t 步要采样的 token；
- $x_{<t}$ ：在此之前已经确定的所有 token；
- $p(x_t \mid x_{<t})$ ：由一次以 $x_{<t}$ 为输入的前向计算给出的条件分布。

第 t 个因子的输入里包含第 $t - 1$ 步的采样结果，这是一条不可消除的数据依赖链。

这不意味着推理完全不能并行。同一步内的矩阵运算、prompt prefill、多个请求之间的 batch 都可以并行。真正受限的是同一条序列的时间依赖，而它恰好让 decode 形成大量“每次只处理一个新 token”的细矩阵运算。第 7 节的投机采样可以理解为：用一个小模型冒险地把这条依赖链向前猜几步，再让大模型并行地验证这些猜测是否成立——它并不消除依赖，只是把依赖的检查并行化了。

核心提示 训练与推理的差别不是“一个用 GPU、一个不用 GPU”，而是可并行维度不同。推理优化的核心任务，是在不破坏自回归依赖的前提下，重新找到共享、复用和批处理机会。

1.4 本章小结

- 推理同时服务产品、评测和训练内采样，因此累计成本巨大。
- TTFT、token latency 与 throughput 衡量不同目标，不能用一个“更快”替代。
- 自回归生成的下一个输入依赖刚采出的 token，导致时间维无法完全并行。
- 本课接下来会沿“性能建模 → 减少状态 → 并行验证 → 动态调度”逐层解决问题。

2 2. 用张量形状与算术强度建立性能语言

2.1 2.1 先统一符号

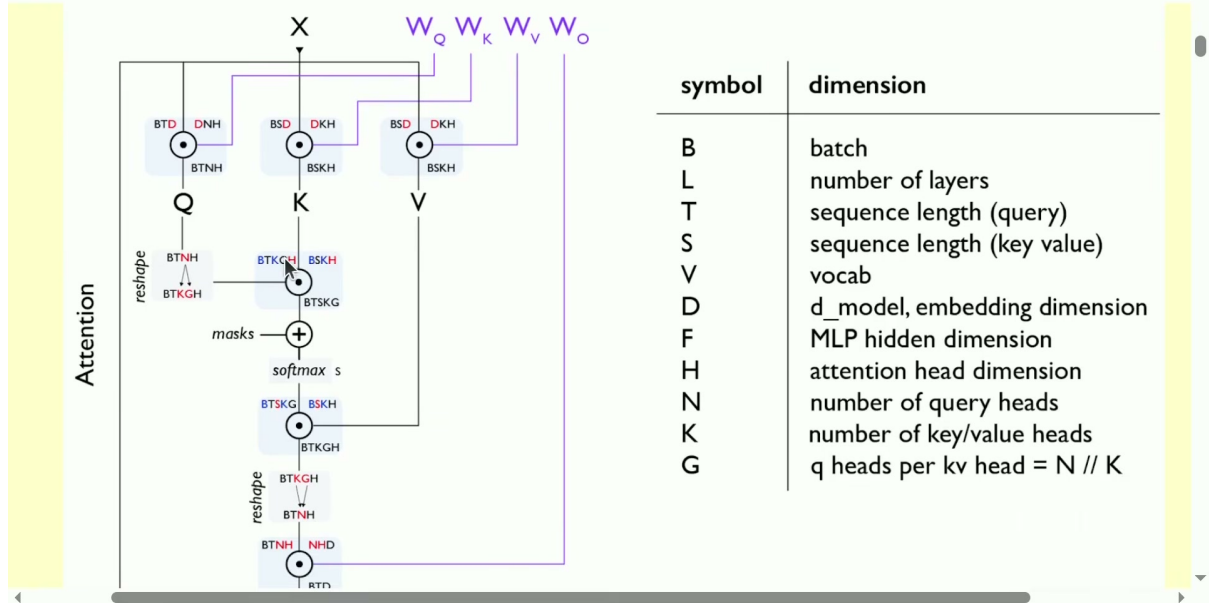
课程使用类似 einops 的紧凑张量记号。以 $BT D \times D H \rightarrow B T H$ 为例， D 在两个操作数中出现、在结果中消失，因此是 contracting dimension； B 若同时出现在两个操作数和结果中，则是 batching dimension，每个 batch 独立计算而不沿 B 求和。读这种记号时只需盯住两件事：谁被求和消掉了（决定 FLOPs），谁只是被平铺（决定可并行度）。

本讲核心符号如下：

- B ：batch size，即并发序列数；
- S ：已经作为条件的历史或输入 token 数；
- T ：本次前向同时处理、为其产生 logits 的目标 token 数；
- D ：model dimension；
- F ：MLP 中间维，课件约定 $F = 4D$ ；
- N ：query head 数；
- K ：key/value head 数，也是 GQA 中的 KV groups 数；
- G ：每个 KV group 服务的 query head 数；
- H ：每个 attention head 的维度；
- L ：Transformer 层数；
- V ：词表大小。

其中 $D = NH$ 、 $N = KG$ 。这组记号很重要，因为后面要区分“所有请求共享的权重”和“每条请求私有的 KV cache”；二者对 batch 的响应完全不同：前者的读取成本被 B 摊薄，后者随 B 线性增长。

- **Batching (blue)** dimensions appear in both operands and stay in result



左侧直接标出 $Q/K/V$ 与 *attention* 中的张量形状，右侧定义 $B, L, T, S, V, D, F, H, N, K, G$ 。

2.2 2.2 算术强度：每搬一个字节，做多少运算

算术强度 (arithmetic intensity) 定义为 FLOPs 与内存传输字节数之比。直觉上，数值越高，同一份数据被复用得越充分；数值越低，设备更可能大部分时间都在等待 HBM 搬数据。它是连接“算法”与“硬件”的桥梁：算法决定分子分母各是多少，硬件决定两者的时间兑换率。

先看 bf16 矩阵乘：输入 X 的形状是 $B \times D$ ，权重 W 的形状是 $D \times F$ ，输出 Y 的形状是 $B \times F$ 。一次点积中的乘法和加法各计一次 FLOP，因此计算量为：

$$\text{FLOPs} = 2BDF.$$

- B ：输入行数或批大小；
- D ：收缩维；
- F ：输出维；
- 系数 2：每项包含一次乘法和一次加法。

推导很直接：输出有 BF 个元素，每个元素是长度为 D 的点积，含 D 次乘与 $D - 1 \approx D$ 次加，合计 $2D$ FLOP，于是总数为 $B \cdot F \cdot 2D = 2BDF$ 。

假设从 HBM 读取 X 、读取 W 、写回 Y ，而 bf16 每个元素占 2 字节，则数据搬运量为：

$$\text{Bytes} = 2BD + 2DF + 2BF.$$

- $2BD$ ：读取输入 X ；
- $2DF$ ：读取权重 W ；

- $2BF$: 写回输出 Y ;
- 每项前的 2: bf16 每元素 2 字节。

把计算量除以搬运量, 就得到精确的一阶算术强度:

$$I = \frac{2BDF}{2BD + 2DF + 2BF} = \frac{BDF}{BD + DF + BF}.$$

- I : 算术强度, 单位 FLOP/byte;
- B, D, F : 含义同上。

当 batch 远小于模型的两个宽度, 即 $B \ll D, F$, 分母主要由权重项 DF 决定, 于是:

$$I \approx B.$$

- I : 近似算术强度;
- B : 同一份权重同时服务的输入行数。

严格地说, 把分母中的 $BD+BF$ 相对 DF 忽略, 要求 $B \ll D$ 且 $B \ll F$; 对 $D = 5120, F = 13824$ 的 13B 模型, B 在数百以内时该近似都相当好。这条近似揭示了 batch 的价值: 同一矩阵被读入一次后, 若能服务更多输入行, 权重读取成本就被摊薄。 $B = 1$ 时退化成矩阵向量乘, 读取整个大权重矩阵, 却只对它使用一次——这是整个推理性能问题的原点。

2.3 2.3 与 H100 的机器平衡点比较

课件采用 H100 bf16 峰值约 989 TFLOP/s、HBM 带宽约 3.35 TB/s。两者相除得到机器的理想平衡点 (ridge point):

$$I_{H100} = \frac{989 \times 10^{12}}{3.35 \times 10^{12}} \approx 295 \text{ FLOP/byte}.$$

- I_{H100} : 课件参数下的理想机器算术强度;
- 989×10^{12} : bf16 峰值浮点运算率;
- 3.35×10^{12} : HBM 每秒可传输的字节数。

这个比值就是 roofline 模型中两条渐近线的交点。对一个执行 F FLOP、搬运 M 字节的 kernel, 其运行时间的下界是:

$$\ell = \max\left(\frac{F}{P_{\text{peak}}}, \frac{M}{W}\right),$$

- ℓ : kernel 的理想运行时间;
- P_{peak} : 峰值计算率 (FLOP/s);
- W : HBM 带宽 (byte/s)。

当 $I = F/M > P_{\text{peak}}/W$ 时第一项占优，时间由算力决定（compute-bound）；反之第二项占优，时间由搬运决定（memory-bound）。把工作负载的强度 I 与机器平衡点 I_{H100} 比较，等价于判断这个 $\max$ 落在哪一支。在刚才 $I \approx B$ 的模型里， $B = 1$ 的推理强度约为 1，离 295 很远——即便把 batch 开到 64，也仍然处于 memory-bound 区域。这正是逐 token decode 的典型形态。

Roofline 一阶估计：给定 FLOPs 与字节数，判断瓶颈并给出时间下界

P_PEAK = 989e12 # H100 bf16 峰值算力 (FLOP/s)

W_HBM = 3.35e12 # H100 HBM 带宽 (byte/s)

```
def roofline(flops: float, bytes_: float) -> tuple[str, float]:
```

```
    t_compute = flops / P_PEAK      # 算力支：算完需要多久
```

```
    t_memory = bytes_ / W_HBM      # 带宽支：搬完需要多久
```

```
    bound = "compute" if t_compute > t_memory else "memory"
```

```
    return bound, max(t_compute, t_memory)
```

decode 一步中一个 $D \times F$ 的权重矩阵， $B=1$

D, F, B = 5120, 13824, 1

flops = 2 * B * D * F

bytes_ = 2 * (B * D + D * F + B * F) # bf16 读写 X、W、Y

bound, t = roofline(flops, bytes_)

print(f"I = {flops/bytes_: .2f} FLOP/byte, {bound}-bound, 下界 {t*1e6: .1f} us")

I ~ 1.00 FLOP/byte, memory-bound, 下界 ~ 42.4 us

注意边界 295 不是所有 H100 程序的固定 batch 阈值。它来自特定精度、标称峰值和极简 HBM accounting。真实 kernel 的融合、cache 命中、通信、launch overhead，以及是否能达到标称峰值，都会移动边界。例如 989 TFLOP/s 是带 sparsity 的标称值，稠密 bf16 峰值约为其一半；但用哪一档峰值只会把平衡点整体平移，不改变“decode 远低于平衡点”的定性结论。

2.4 为什么“memory-bound”会决定后面的优化方向

如果瓶颈是计算，减少 FLOPs 或增加更高效的矩阵乘最直接；如果瓶颈是内存，单纯减少少量算术未必有用，更有效的路线是：

- 少读权重：量化、剪枝；
- 少读请求私有状态：GQA、MLA、CLA、局部或稀疏注意力；
- 让一次权重读取服务更多 token：batch、continuous batching、投机验证；
- 少浪费物理内存和重复前缀：PagedAttention 与 prefix sharing。

这也是为什么本讲先花很长时间推导性能模型，再介绍具体技术。没有模型，就无法判断优化究竟作用在哪个瓶颈上：一个把 FLOPs 减半的“高效 attention”，在 memory-bound 区域可能几乎不改变墙钟时间。

三个典型算子的强度对比 把 2.2 的方法推广到本讲关心的三类运算，可以一次性看清 decode 为什么处处受带宽约束。记序列历史长度为 S 、模型维 D 。

decode 阶段的线性层 ($B = 1$ ，单个 token 通过 $D \times D$ 权重矩阵)：

$$\frac{\text{FLOPs}}{\text{Bytes}} = \frac{2D^2}{2D^2 + 2D + 2D} \approx 1 \text{ FLOP/byte},$$

- 分子 $2D^2$ ：矩阵向量乘的乘加次数，量级 $O(D^2)$ ；
- 分母 $2D^2 + 4D$ ：读取权重 D^2 个 bf16 元素，加读写各一个 D 维向量，量级同为 $O(D^2)$ 。

FLOPs 与字节同阶，系数相消后强度是 $O(1)$ 的常数，与 D 无关——**模型越大，每一步要搬的字节和要做的运算同比放大，强度不变，永远困在 memory-bound 区。**

decode 阶段的 attention ($B = 1$ ，一个新 query 对 S 个历史 token)： $QK^\top$ 与加权求和 AV 各约 $2SD$ FLOP，合计 $O(SD)$ ；读取历史 K、V 各 SD 个元素，字节也是 $O(SD)$ 。两者相除得 $I = S/(S + 1) < 1$ (3.5 节会完整推导)。

prefill 阶段 ($T = S$ 个 token 一起过)：attention 的 FLOPs 为 $O(S^2D)$ (每个 query 都要看 S 个 key)，而 KV 只需读一次共 $O(SD)$ 字节，强度升到约 $S/2$ ；线性层则因 B 换成 S 而强度约 S 。

运算	FLOPs	字节数	算术强度	对 295 的位置
decode 线性层 ($B=1$)	$O(D^2)$	$O(D^2)$	≈ 1	深在 memory-bound 区
decode attention	$O(SD)$	$O(SD)$	$S/(S + 1) < 1$	深在 memory-bound 区
prefill attention	$O(S^2D)$	$O(SD)$	$\approx S/2$	$S > 590$ 时越界转 compute-bound
prefill/decode 线性层 (B 行)	$2BDF$	$\approx 2DF$	$\approx B$	B 要到数百才接近平衡点

这张表浓缩了整堂课的力学结构：prefill 是“算力活”，decode 是“搬运活”；优化 decode 的一切手段，本质都是减少每步要搬的字节，或让一次搬运服务更多 token。

2.5 本章小结

- 算术强度等于 FLOPs/byte，用于判断工作负载更接近计算瓶颈还是带宽瓶颈。
- 对大权重、小 batch 的矩阵乘，强度近似等于 batch size。
- H100 课件示例的机器平衡点约为 295 FLOP/byte，单 token 矩阵向量乘远低于它。
- 后续技术虽名称不同，本质都在减少搬运、增加复用或重排可并行工作。

3 从朴素生成到 KV cache：计算换成了状态

3.1 为什么朴素自回归会做大量重复工作

最直观的生成循环是：把完整 prompt 送入 Transformer，采一个 token，把它拼到历史末尾，再把更长的整段历史重新送入 Transformer。算法正确，却重复计算了所有旧 token 的 K/V 和旧 token 之间的 attention。

若长度为 t 的 full attention 成本为二次量级，把所有生成步累加起来，朴素全过程达到三次量级：

$$\sum_{t=1}^T O(t^2) = O(T^3).$$

- t ：某个生成步已经拥有的前缀长度；
- T ：最终生成长度；
- $O(t^2)$ ：该步重新对完整前缀做 attention 的数量级。

这里的求和可以用积分估出系数直觉： $\sum_{t=1}^T t^2 \approx T^3/3$ 。也就是说，朴素实现不仅渐近阶高，而且常数也可观——生成 1000 个 token 就要付出约 $10^9/3$ 量级的 attention 单元运算，其中绝大多数是在重算与上一步几乎相同的东西。

因果 attention 提供了关键复用机会。追加新 token 后，历史位置不能看到未来，因此它们原先算出的 K/V 不会被新 token 改写。用数学语言说：位置 i 的 key/value 只依赖 $x_{\leq i}$ 与模型权重，不依赖 $x_{>i}$ ；于是 x_{t+1} 的到来不会改变任何 $i \leq t$ 的 K/V。只要把每层历史 K/V 留在内存里，后续步骤就不必再计算旧前缀。

3.2 KV cache 的收益与代价

KV cache 把旧 token 的 key 和 value 保存到 HBM。新 token 到来时，只计算自己的 Q/K/V，用新 query 读取所有历史 K/V，然后把新 K/V 追加到 cache。

它带来两个同时成立的结果：

- 计算收益：避免历史投影和历史—历史 attention 的重复计算，整个生成过程的 attention 从朴素三次量级降到二次量级——第 t 步只做 $O(td)$ 的新工作（一个新 query 对 t 个历史 KV），求和 $\sum_t O(td) = O(T^2d)$ ；
- 内存代价：每层、每个历史 token 的 K/V 必须长期驻留，并在每个 decode step 被读取。第 t 步要读的 cache 是 $O(td)$ 字节，这意味着 decode 越到后期每一步越慢，cache 同时占用容量与带宽两种资源。

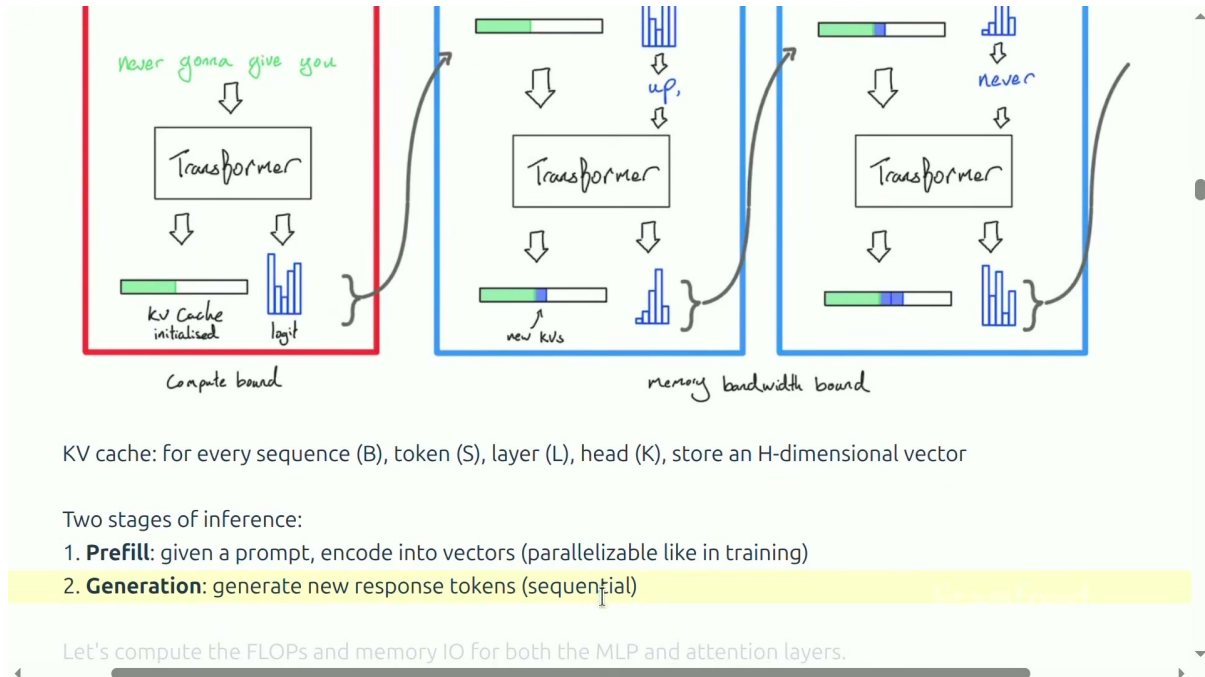
核心提示 KV cache 没有让推理“免费”。它只是把主要问题从重复计算转换成 cache 容量和 HBM 带宽。本讲后半的大多数架构优化，都是在设法减少这份状态。

3.3 Prefill 与 decode 是两个不同阶段

Prefill 一次看到整个 prompt，可以沿 token 维并行计算，并填充所有层的 KV cache。它的形态接近训练，长 prompt 往往形成足够大的矩阵运算。由 2.4 小节的分析，prefill 中线性层的强度约为 S 、attention 的强度约为 $S/2$ ，当 prompt 达到数百 token 时就已经贴近甚至越过 H100 的机器平衡点，因此 prefill 通常是 compute-bound 的，优化它的手段是更强的算力、更好的 kernel 与算子融合。

Decode / generation 每次只得到一个 token，读取已有 KV cache、生成下一个 token、再把新 K/V 追加进去。它沿时间串行，且上下文越长，每步要读的 cache 越大，强度始终停留在 $O(1)$ 量级，因此 decode 通常是 memory-bound 的，优化它的手段是少搬字节。

在后续公式里， S 表示已作为条件的 token 数， T 表示本次同时为其产生 logits 的 token 数：prefill 取 $T = S$ ，逐 token decode 取 $T = 1$ 。这一对阶段划分也解释了为什么在线服务常把 prefill 和 decode 分开调度甚至分开部署：它们争的是两种不同的硬件资源。



红框 prefill 一次填充 KV cache，蓝框 generation 每步追加新 K/V；底部明确对比“可并行的 prefill”与“串行的 generation”。

3.4 MLP 的算术强度

课程只统计 gated MLP 的三个矩阵乘：up、gate、down。三次矩阵乘各需要 $2BTDF$ FLOP，因此总计算量为：

$$\text{FLOPs}_{\text{MLP}} = 6BTDF.$$

- B ：并发序列数；
- T ：本次同时处理的 token 数；
- D ：模型维；
- F ：MLP 中间维；
- 系数 6：三次矩阵乘，每次乘加计 2 FLOP。

逐项核对：up 与 gate 都是 $BT D \times D F \rightarrow B T F$ ，各 $2BTDF$ ；down 是 $B T F \times F D \rightarrow B T D$ ，收缩维是 F 、输出维是 D ，也是 $2BTDF$ 。三者相加即得上式。

按课件的未融合朴素 accounting，读写输入/输出、中间激活和三组权重的总字节数为：

$$\text{Bytes}_{\text{MLP}} = 4BT D + 4B T F + 6D F.$$

- $4BT D$ ：输入与输出的 bf16 读写；

- $4BTF$: 两个中间激活的写入;
- $6DF$: 三个 bf16 权重矩阵的读取。

当 token batch BT 远小于模型宽度时, 权重读取主导, 强度近似为:

$$I_{\text{MLP}} \approx BT.$$

- I_{MLP} : MLP 的近似算术强度;
- B : 并发请求数;
- T : 一次并行处理的 token 数。

因此 prefill 可以靠大 $B \times S$ 获得较高强度; decode 时 $T = 1$, 只剩 batch B 能摊薄公共权重。这个结论值得记住: **对权重而言, batch 是免费的午餐——同一趟 HBM 读取, 服务 1 条请求和服务 64 条请求的搬运成本完全相同。**

3.5 普通 MHA attention 的算术强度

在采用 FlashAttention 式“不把完整 attention 矩阵落回 HBM”的假设下, $QK^\top$ 和 $\text{softmax}(A)V$ 两次矩阵乘合计:

$$\text{FLOPs}_{\text{attn}} = 4BSTD.$$

- B : 并发序列数;
- S : 历史 token 数;
- T : 本次 query token 数;
- D : 所有 query heads 合计的模型维;
- 系数 4: 两次矩阵乘, 每次乘加计 2 FLOP。

推导: $QK^\top$ 输出 $B \times T \times S$ 个注意力 logits, 每个是长 D (把所有 query heads 拼起来看) 的点积, 计 $2D$ FLOP, 共 $2BSTD$; 同理 $\text{softmax}(A)V$ 输出 $B \times T \times D$, 每个输出分量是对 S 项的加权和, 计 $2S$ FLOP, 共 $2BTSD$ 。两项相加得 $4BSTD$ 。这里把 N 个 head 的维度 H 已经归并进 $D = NH$, softmax 本身的 $O(BTS)$ 指数与归一化相对矩阵乘是高阶小量, 按惯例忽略。

课件按普通 MHA 的 K/V 总宽度 D 计算, 读 Q/K/V 并写输出的字节数为:

$$\text{Bytes}_{\text{attn}} = 4BSD + 4BTD.$$

- $4BSD$: 读取历史 K 与 V;
- $4BTD$: 读取 Q 并写回输出;
- 这里隐含 MHA, 即 KV 总宽度等于 D 。

因此算术强度是:

$$I_{\text{attn}} = \frac{ST}{S+T}.$$

- I_{attn} : 普通 MHA attention 的算术强度;
- S : 历史 token 数;
- T : 本次 query token 数。

prefill 取 $T = S$, 得到:

$$I_{\text{prefill,attn}} = \frac{S}{2}.$$

- $I_{\text{prefill,attn}}$: prefill attention 强度;
- S : prompt 长度。

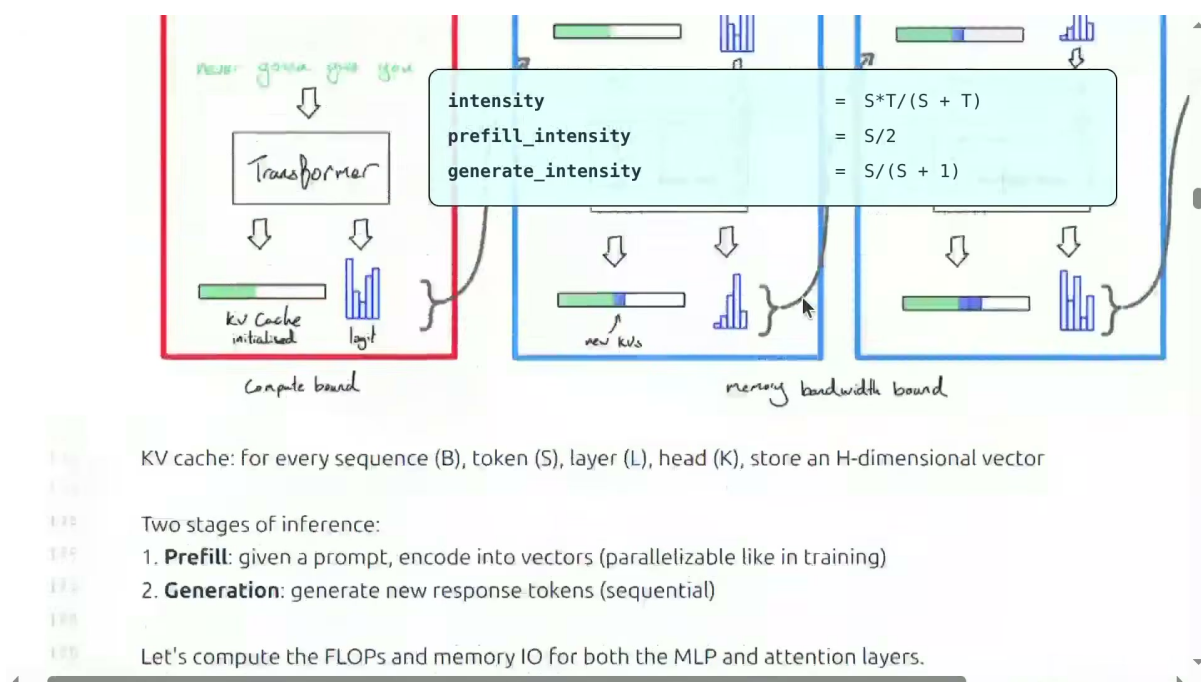
decode 取 $T = 1$, 得到:

$$I_{\text{decode,attn}} = \frac{S}{S+1} < 1.$$

- $I_{\text{decode,attn}}$: 逐 token decode attention 强度;
- S : 当前历史长度。

注意这个值的渐近行为: 当 $S \rightarrow \infty$ 时它单调趋近于 1, 永远不会超过 1——上下文越长, decode attention 的强度越贴近 1 但仍深陷 memory-bound 区, 且每一步要搬的绝对字节数 $4BSD$ 还在线性增长。

最关键的是, attention 强度里没有 B 。MLP 权重对所有请求相同, 增大 batch 会复用同一份权重; 每条请求的 KV cache 却不同, batch 增大时计算和要读取的 KV 同比增加, 所以无法靠 batch 改善普通 MHA decode attention 的强度。这一点是第 5 节所有 KV 压缩技术的动机: 既然 batch 摊不薄私有状态, 那就只能把私有状态本身做小。



上方调试器给出 $ST/(S+T)$ 、 $S/2$ 与 $S/(S+1)$ ；背景的三条生成链各有自己的 KV cache，说明 $batch$ 不会形成跨请求 KV 复用。

注意边界 < 1 不是“所有 attention 架构永远不可改善”。这段推导明确采用 MHA 的 KV 宽度。GQA、MLA 和稀疏注意力正是通过减小要读的 KV 宽度或数量来改变分母。

3.6 本章小结

- 因果性使历史 K/V 可复用， KV cache 将全过程 attention 从三次量级降到二次量级。
- Prefill 能沿 prompt 并行，常更接近 compute-bound；decode 一次只有一个新 token，常 memory-bound。
- MLP decode 可借 batch 摊薄公共权重；普通 MHA attention 的 KV 是请求私有的，batch 不能提高其强度。
- KV cache 因而同时是推理加速的基础，也是容量与带宽瓶颈的来源。

4 4. 从简化性能模型看 latency-throughput 权衡

4.1 参数量与 KV cache 大小

课程用一个简化 Transformer 模型计算 Llama 2 13B 在 H100 上的理想性能。参数量由 embedding、gated MLP 和 attention 投影构成：

$$P = 2VD + 3DFL + (2DNH + 2DKH)L.$$

- P ：参数个数；
- V ：词表大小；
- D ：模型维；
- F ：MLP 中间维；
- L ：层数；
- N ：query head 数；
- K ： KV head 数；
- H ：单 head 维度；
- $2VD$ ：课件按输入、输出 embedding 各一份计算，未假设 weight tying。

逐项来源：gated MLP 每层三个 $D \times F$ 矩阵共 $3DF$ ；attention 每层四个投影—— $W_Q : D \rightarrow NH$ 、 $W_K, W_V : D \rightarrow KH$ 、 $W_O : NH \rightarrow D$ ，合计 $DNH + 2DKH + NHD = 2DNH + 2DKH$ （用到 $D = NH$ ）。bias、LayerNorm 参数相对矩阵是高阶小量，忽略。

bf16 参数每个占 2 字节，因此参数内存为：

$$M_{\text{param}} = 2P.$$

- M_{param} : 参数占用字节数;
- P : 参数个数;
- 系数 2: bf16 每参数 2 字节。

一条长度为 S 的序列, 在 L 层、每层 K 个 KV heads、每 head 维度为 H 时, 其 cache 为:

$$M_{\text{KV/seq}} = S(KH)L \times 2 \times 2.$$

- $M_{\text{KV/seq}}$: 单条序列的 KV cache 字节数;
- S : 缓存 token 数;
- K : KV head 数;
- H : 每个 head 的维度;
- L : 层数;
- 第一个 2: key 与 value 两份向量;
- 第二个 2: bf16 每元素 2 字节。

4.2 4.2 带宽下界模型

在“decode memory-bound、每步读完相关参数与 KV、计算和通信完美重叠、忽略所有 overhead”的强假设下, 总内存、单步延迟和吞吐写成:

$$M = M_{\text{param}} + BM_{\text{KV/seq}}, \quad \ell = \frac{M}{W}, \quad \text{throughput} = \frac{B}{\ell}.$$

- M : batch 的总参数与 KV 字节数;
- M_{param} : 参数字节数;
- $M_{\text{KV/seq}}$: 单序列 KV 字节数;
- B : 并发序列数;
- ℓ : 理想 decode step 延迟;
- W : HBM 带宽;
- B/ℓ : 每步并行生成 B 个 token, 所以得到总体 tokens/s。

这个模型背后的逻辑链值得展开。既然 decode 的每个算子都深陷 memory-bound 区, 每一步的墙钟时间就由“这一步必须从 HBM 搬进多少字节”决定。一个 decode step 需要: 全部权重各读一遍 (所有请求共享, 计一次), 外加每条请求各自的全部历史 KV 各读一遍 (私有, 计 B 次)。两者之和除以带宽, 就是单步延迟的带宽下界:

$$\ell \geq \frac{M_{\text{param}} + B \cdot M_{\text{KV/seq}}}{W}.$$

吞吐则是每步产出的 B 个 token 除以这一步的耗时。把三个式子联立, 还能得到一个更有意思的形式: 当 B 较小、 M_{param} 主导时, $\text{throughput} \approx BW/M_{\text{param}}$, 吞吐随 batch 近似线性增长; 当

B 大到 KV 主导时, $\text{throughput} \approx W/M_{\text{KV/seq}}$ 趋于饱和, 不再随 batch 增长——饱和值恰是“带宽除以单条序列每步要搬的 KV 字节数”, 这把第 5 节 KV 压缩的收益直接翻译成了吞吐上限。

这不是实测预测, 而是 bandwidth-bound 的理想下界。它忽略 prefill、kernel launch、通信、调度、cache 命中, 以及 batch 足够大后重新转为 compute-bound 的可能性。

4.3 Llama 2 13B / H100 的数值

官方配置采用 $S = 1024, D = 5120, F = 13824, N = 40, H = 128, L = 40, V = 32000$, H100 带宽 3.35 TB/s。我们先把 4.1 的两个总量亲手算出来, 再与课件调试器对拍。

参数量 (MHA, $K = N = 40$):

$$\begin{aligned} 2VD &= 2 \times 32000 \times 5120 = 3.28 \times 10^8, \\ 3DFL &= 3 \times 5120 \times 13824 \times 40 = 8.493 \times 10^9, \\ (2DNH + 2DKH)L &= (2 \times 5120 \times 5120 + 2 \times 5120 \times 5120) \times 40 \\ &= 1.049 \times 10^8 \times 40 = 4.194 \times 10^9, \end{aligned}$$

合计 $P = 0.328 + 8.493 + 4.194 \approx 13.015 \times 10^9$, 即 13.015B, 与模型名称吻合; bf16 参数内存 $M_{\text{param}} = 2P \approx 26.03 \text{ GB}$ 。

单序列 KV cache:

$$M_{\text{KV/seq}} = 1024 \times (40 \times 128) \times 40 \times 2 \times 2 = 8.39 \times 10^8 \text{ B} \approx 0.839 \text{ GB}.$$

$B = 1$ 的理想单步延迟与吞吐:

$$\ell = \frac{26.03 + 0.839}{3.35 \times 10^3} \text{ s} = \frac{26.87 \text{ GB}}{3.35 \text{ GB/ms}} \approx 8.02 \text{ ms}, \quad \text{throughput} = \frac{1}{0.00802} \approx 124.7 \text{ tok/s}.$$

$B = 64$: 总内存 $26.03 + 64 \times 0.839 = 79.72 \text{ GB}$, $\ell = 79.72/3.35 \approx 23.80 \text{ ms}$, 吞吐 $64/0.02380 \approx 2689 \text{ tok/s}$ 。可以看到吞吐提升 21.6 $\times$, 远小于 batch 的 64 $\times$ ——差额正是每条新请求带来的 KV 读取代价。

按源码公式重算:

```

336
337 Instantiate latency and throughput for llama 2 7B
338 config = llama2_13b_config()
339 stats = compute_transformer_performance_stats(
340
341 # Batch size 1
342 b1 = stats.substitute(B, 1)
343
344 # Batch size 64
345 b64 = stats.substitute(B, 64)
346 Result: worse latency, better throughput
347
348 # Batch size 256
349 b256 = stats.substitute(B, 256)
350 Result: even worse latency, even better throughput
351 h100_memory = 80e9 # H100 memory in bytes
352 assert b256.memory > h100_memory # Doesn't fit in memory!
353
354 Result: doesn't fit into memory and throughput gains are diminishing too...
355
356 What increasing batch size does:

```

b1	=	num_params : 13,015,449,600 memory : 26,869,760,000 latency : 0.0080 throughput : 124.6755
b64	=	num_params : 13,015,449,600 memory : 79,717,990,400 latency : 0.0238 throughput : 2689.4807
b256	=	num_params : 13,015,449,600 memory : 240,779,200,000 latency : 0.0719 throughput : 3561.7685

课程调试器同时显示三个 *batch* 的参数量、总内存、*latency* 与 *throughput*; $B = 256$ 的 240.78GB 超过 80GB H100。

架构	Batch	参数量	参数 bf16	每序列 KV	总内存	理想单步延迟	理想吞吐
MHA, $K = 40$	1	13.015B	26.03 GB	0.839 GB	26.87 GB	8.02 ms	124.7 tok/s
MHA, $K = 40$	64	13.015B	26.03 GB	0.839 GB	79.72 GB	23.80 ms	2689 tok/s
MHA, $K = 40$	256	13.015B	26.03 GB	0.839 GB	240.78 GB	71.87 ms	3562 tok/s
GQA, $K = 8$	64	11.338B	22.68 GB	0.168 GB	33.41 GB	9.97 ms	6417 tok/s
GQA, $K = 8$	256	11.338B	22.68 GB	0.168 GB	65.63 GB	19.59 ms	13068 tok/s

GQA 两行也照例验算。 $K = 8$ 时 attention 投影参数变为 $(2DNH + 2DKH)L = (52428800 + 10485760) \times 40 \approx 2.517 \times 10^9$ ，总参数 $0.328 + 8.493 + 2.517 \approx 11.338 \times 10^9$ ；每序列 KV = $1024 \times (8 \times 128) \times 40 \times 4 \approx 0.168$ GB。 $B = 64$ 时总内存 $22.68 + 64 \times 0.168 = 33.41$ GB, $\ell = 9.97$ ms, 吞吐 6417 tok/s; $B = 256$ 时总内存 65.63 GB——注意它已经低于 MHA 的 $B = 64$ ，而吞吐高出近 5 $\times$ 。

KV cache 与带宽下界计算器：复现上表任意一行

```

def perf_model(S, D, F, N, K, H, L, V, B, W=3.35e12, bytes_per=2):
    # 参数量: embedding + MLP + attention 投影
    P = 2 * V * D + 3 * D * F * L + (2 * D * N * H + 2 * D * K * H) * L
    m_param = P * bytes_per # 参数字节数
    m_kv = S * K * H * L * 2 * bytes_per # 单序列 KV 字节数
    M = m_param + B * m_kv # 单步要搬的总字节
    latency = M / W # 带宽下界延迟
    return P / 1e9, m_kv / 2**30, M / 2**30, latency * 1e3, B / latency

```

Llama 2 13B, MHA, B=64

```

p, kv, m, lat, thr = perf_model(1024, 5120, 13824, 40, 40, 128, 40, 32000, 64)
print(f"{p:.3f}B 参数, KV/seq {kv:.3f}GB, 总 {m:.2f}GB, "

```

```
f"{lat:.2f}ms, {thr:.0f} tok/s")
# 13.015B 参数, KV/seq 0.781GiB, 总 74.25GiB(=79.72GB), 23.80ms, 2689 tok/s
```

这些数值展示了三个规律：

1. batch 增大后，固定参数读取被更多 token 分摊，吞吐显著提高；
2. 每条请求又会新增自己的 KV cache，因此单步延迟与总内存随 batch 上升；
3. MHA 的 batch 256 远超 80GB H100，而 GQA 把 KV heads 从 40 降为 8 后，才让这个 batch 在简化模型中放得下——这给出了一个可操作的容量规划公式：给定显存预算 M_{HBM} ，最大活动 batch 约为

$$B_{\max} \approx \frac{M_{\text{HBM}} - M_{\text{param}}}{M_{\text{KV/seq}}},$$

对 GQA 配置代入 $(80 - 22.68)/0.168 \approx 341$ ，即理想情况下约三百余条并发序列；分子上每省出一 GB、分母上每缩小一倍， $B_{\max}$ 都会相应放大。这正是“省 KV 就是省显存，省显存就是吞吐”的定量版本。

讲者用公交车解释 latency-throughput 权衡：等一辆大车可能让单个乘客更慢，但一趟能运走更多人。对交互请求，较小 prefill batch 有助于 TTFT；对 decode 或批处理，较大活动 batch 有助于吞吐。

注意边界 课件源码在 GQA $K = 8, B = 64$ 后写着“worse latency”。若与同 batch 的 MHA 比，模型给出的延迟其实从 23.80 ms 降到 9.97 ms；若与最初 batch 1 的 MHA 比，9.97 ms 才略差于 8.02 ms。讲义采用数值并明确比较对象，不复述含糊的形容词。

4.4 复制模型与分片模型

如果有足够设备，启动 M 份模型副本，每个副本的单请求延迟不变，而总吞吐可近似增加 M 倍。更复杂的做法是把模型和 KV cache 分片到多设备；它允许单个超大模型运行，却引入设备间通信和同步，因此不能沿用“只看单卡 HBM”的公式——tensor parallel 的每层 all-reduce 把 NVLink/网络带宽也变成了新的潜在瓶颈，roofline 分析必须对每个互联层分别做。

TTFT 则主要受 prefill 和排队影响。服务系统常需要分别调度 prefill 与 decode：前者用较小 batch 减少首 token 等待，后者用持续填充的较大 batch 提高稳态吞吐。这种 prefill/decode 分离 (disaggregation) 的思路，本质上就是承认两个阶段落在 roofline 的不同区域，应该用不同的资源配比去服务。

4.5 本章小结

- 参数是所有请求共享的固定成本，KV cache 是随 batch 和上下文增长的需求私有成本。
- 增大 batch 能提高吞吐，却会增加 KV 内存与单步延迟，并最终遇到容量上限。
- GQA 的价值可直接从性能模型看到：减少 KV heads 后，既降低单序列 cache，也允许更大的活动 batch。
- 简化公式是推理系统的思考工具，不应伪装成完整的端到端基准。

5. 5. 压缩 KV cache：沿不同轴减少每步搬运

5.1 5.1 一张地图：宽度、层数与历史长度

普通 MHA 的 KV cache 可以粗略看成四个维度的乘积：历史 token 数 S 、层数 L 、KV head 数 K 和每 head 宽度 H 。于是压缩方法可按“动了哪一轴”分类：

- GQA/MQA：减少 KV heads，在 query heads 之间共享；
- MLA：缓存低维 latent，需要时再恢复多头 K/V；
- CLA：在相邻或指定 layers 之间共享 K/V；
- local / sliding-window attention：只保留有限的历史 token；
- 稀疏注意力：压缩历史、建立便宜索引，再选出少量相关条目。

共同因果链是：更小的 cache $\rightarrow$ 每个 decode step 搬运更少字节 $\rightarrow$ 在 memory-bound 区域降低 latency、提高 throughput，并释放显存容纳更大的 batch。但压缩常会牺牲表达力或增加投影、索引成本，所以最后一环永远是重新测 accuracy 和端到端性能。

各路线每层每 token 缓存的统一公式 把“每层每 token 缓存多少字节”写成统一形式，所有路线的差别就只剩一个“缓存宽度”因子。记每个元素占 b 字节 (bf16 为 2)，key 与 value 共两份：

方案	每 token 每层缓存	相对 MHA 比例	Llama 2 13B 实例 ($N=40, H=128, L=40, b=2$)
MHA	$2bNH$	1	$2 \times 2 \times 40 \times 128 = 20480$ B/层/token，全模型 ≈ 0.82 MB/token
GQA (K 组)	$2bKH$	$K/N = 1/G$	$K=8$: 4096 B/层/token，全模型 ≈ 0.16 MB/token
MQA	$2bH$	$1/N$	512 B/层/token，全模型 ≈ 20 KB/token
MLA	$b(C + C_{\text{rope}})$	$(C + C_{\text{rope}})/(2NH)$	DeepSeek-V2: $C=512, C_{\text{rope}}=64$ ，576 元素 ≈ 1.15 KB/层/token
CLA (r 层共享)	$2bKH/r$	$K/(rN)$	在 GQA 基础上再按共享度 r 缩减
Sliding window (窗口 w ，上下文 S)	$2bKH \cdot \min(w, S)/S$ (摊到每 token)	$\approx w/S$	$w=4096, S=128K$ 时约 $1/32$

- C ：MLA 缓存的 latent 维度；
- C_{rope} ：MLA 中为位置编码单独保留的维度；
- r ：CLA 中共享同一份 K/V 的层数；
- w ：滑动窗口宽度。

MLA 的比例写法略有不同：它缓存的是 key 与 value 共用的单一 latent，因此没有因子 2。读这张表的方式是：每一行都在回答“decode 一步，每个历史 token 要从 HBM 搬多少字节”，把它乘上 $B \times S$ 就是 4.2 节带宽下界模型里 KV 那一项。

5.2 5.2 MQA 与 GQA：在 heads 之间共享

MHA 为每个 query head 保留独立的 key/value head，即 $K = N$ ；MQA 令所有 query heads 共用一组 K/V，即 $K = 1$ ；GQA 取两者之间，让 $G = N/K$ 个 query heads 共享一个 KV head。

单序列 KV cache 的字节数可以写成：

$$M_{KV} = S \cdot KH \cdot L \cdot 2 \cdot b.$$

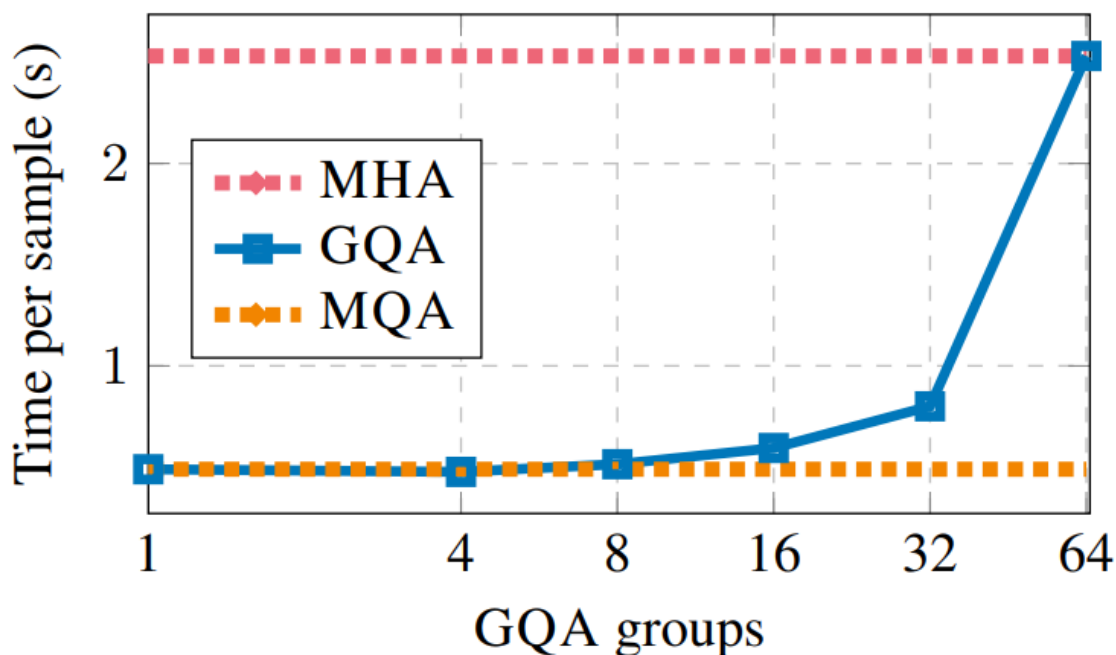
- M_{KV} ：单序列 KV cache 字节数；
- S ：历史 token 数；
- K ：KV head 数；
- H ：每个 head 的维度；
- L ：层数；
- 第一个 2：key 与 value 两份；
- b ：每个元素的字节数，bf16 时为 2。

因此 GQA 相对 MHA 的 cache 比例是：

$$\frac{M_{GQA}}{M_{MHA}} = \frac{K}{N} = \frac{1}{G}.$$

- M_{GQA} ：GQA 的 cache 大小；
- M_{MHA} ：MHA 的 cache 大小；
- K ：GQA 的 KV heads；
- N ：query heads；
- $G = N/K$ ：每组共享的 query heads 数。

值得注意的是，GQA 不仅压缩 cache，还顺带减少了参数： W_K, W_V 从 $D \times NH$ 缩小到 $D \times KH$ ，4.3 节已经算出这让 13B 模型“瘦”到 11.3B，参数读取成本同步下降。实现上，共享并不是把 K/V 复制 G 份（那就失去了压缩意义），而是让 G 个 query head 在 kernel 内读取同一份 K/V，搬运量按一份计。



横轴 *groups* 由 1 增至 64, *GQA* 从接近 *MQA* 逐渐回到 *MHA*; 组数更少意味着更多 *query heads* 共享 K/V 。

固定 batch、把 $K = 40$ 改为 $K = 8$ 时, 单条请求要搬的 KV 直接下降, 所以 latency 和 throughput 可以一起改善。之后再吧 batch 从 64 提到 256, 才重新出现“更高吞吐换更差单步延迟”的权衡。这里必须把“架构压缩”和“增大 batch”两次操作分开: 前者在 latency-throughput 平面上是 Pareto 改善 (两个指标同时变好), 后者只是沿权衡曲线移动。

Model	T_{infer}	Average	CNN	arXiv	PubMed	MediaSum	MultiNews	WMT	TriviaQA
	s		R_1	R_1	R_1	R_1	R_1	BLEU	F1
MHA-Large	0.37	46.0	42.9	44.6	46.2	35.5	46.6	27.7	78.2
MHA-XXL	1.51	47.2	43.8	45.6	47.5	36.4	46.9	28.4	81.9
MQA-XXL	0.24	46.6	43.0	45.0	46.9	36.1	46.5	28.5	81.3
GQA-8-XXL	0.28	47.1	43.5	45.4	47.7	36.3	47.2	28.4	81.6

在这组论文实验中, *GQA-8-XXL* 的平均质量 47.1 接近 *MHA-XXL* 的 47.2, 而推理时间 0.28s 接近 *MQA-XXL* 的 0.24s。

注意边界 一张表不能证明 *GQA* 普遍无损。讲者紧接着提醒, 所有非纯数学结果都要结合模型和实验设置看; 后面的 DeepSeek 表里, *GQA* 相对 *MHA* 就出现了明显掉点。

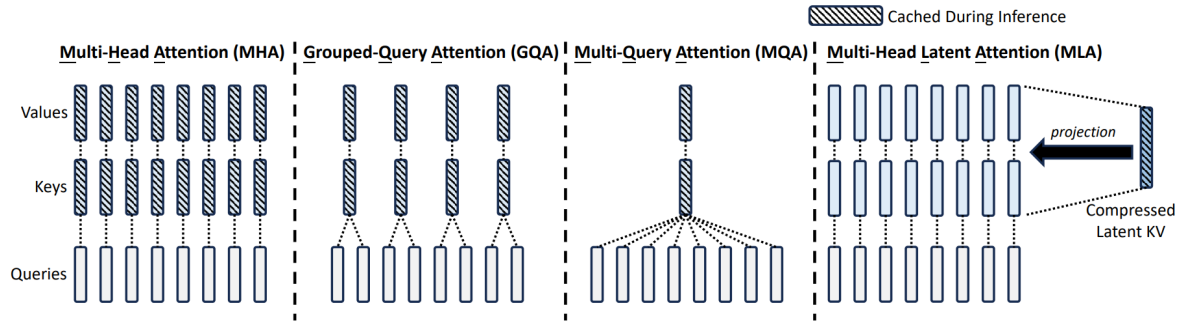
5.3 5.3 MLA: 缓存 latent, 而不是直接缓存完整 K/V

MLA 不只减少 heads, 而是把当前隐藏状态压到低维 latent, cache 中保存 latent, 需要 attention 时再投影出 K/V :

$$c = W_c h, \quad K = W_K c, \quad V = W_V c.$$

- h : 当前 token 的隐藏状态;
- W_c : 向低维 cache 空间投影的矩阵;
- c : 维度为 C 的缓存 latent;
- W_K, W_V : 从 latent 恢复 key/value 的投影;
- K, V : 真正参与 attention 的多头表示。

这个设计的代数本质是低秩分解：完整投影 $W_K^{(\text{full})}h$ 被拆成 $W_K W_c h$ ，把 $D \times NH$ 的大矩阵换成 $D \times C$ 与 $C \times NH$ 两个小矩阵。只要 $C \ll NH$ ，缓存一个 C 维向量就比缓存 NH 维的完整 K 、 V 便宜得多——而且 key 与 value 共用同一份 latent，又省掉了因子 2。



斜线填充表示推理时真正缓存的部分。MLA 只缓存右侧 *compressed latent*，再投影为多头 K/V 。

课上以 DeepSeek-V2 为例：普通多头宽度 $NH = 16384$ ，latent 只有 512 维；由于 RoPE 位置部分不能简单与内容压缩完全合并，还要另保留 64 维，总缓存宽度为 576。直觉上，RoPE 对 key 施加的是依赖位置的旋转，若把整个 $W_K W_c$ 合并成单一大矩阵，旋转就无法被吸收进静态权重，因此工程上把 key 分成“内容相关低秩部分”与“位置相关小维度部分”分别缓存。讲义只保留课程给出的设计直觉：内容相关低秩部分与位置相关部分分开处理；视频没有展开完整代数证明。

先看简单共享的代价：

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	37.0
MMLU (Acc.)	5-shot	37.9	41.2	45.2
C-Eval (Acc.)	5-shot	30.0	37.7	42.9
CMMLU (Acc.)	5-shot	34.6	38.4	43.5

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

DeepSeek-V2 的 Dense 7B 对比中，MHA 在 BBH、MMLU、C-Eval 与 CMMLU 上明显高于 GQA/MQA，说明共享 K/V 的质量结论依赖具体设置。

再看 MLA 的结果：

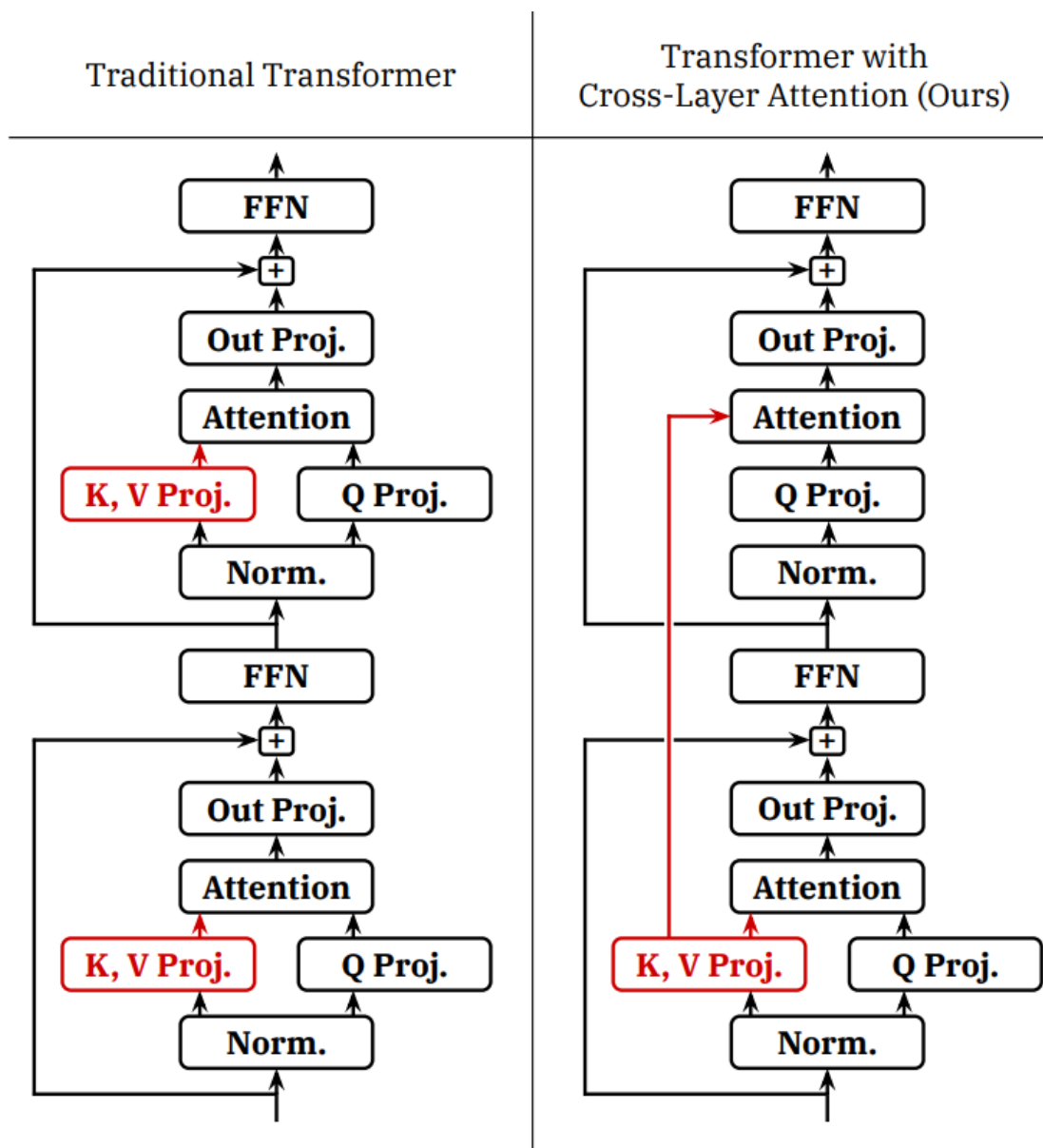
Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	39.0	46.6	50.7
MMLU (Acc.)	5-shot	48.7	50.0	57.5	59.0
C-Eval (Acc.)	5-shot	51.6	50.9	57.9	59.2
CMMLU (Acc.)	5-shot	52.3	53.4	60.7	62.5

Small/Large MoE 的 *cache/token* 分别从 *110.6K/860.2K* 元素降到 *15.6K/34.6K*；多数指标略升，但并非每个单项都更高。

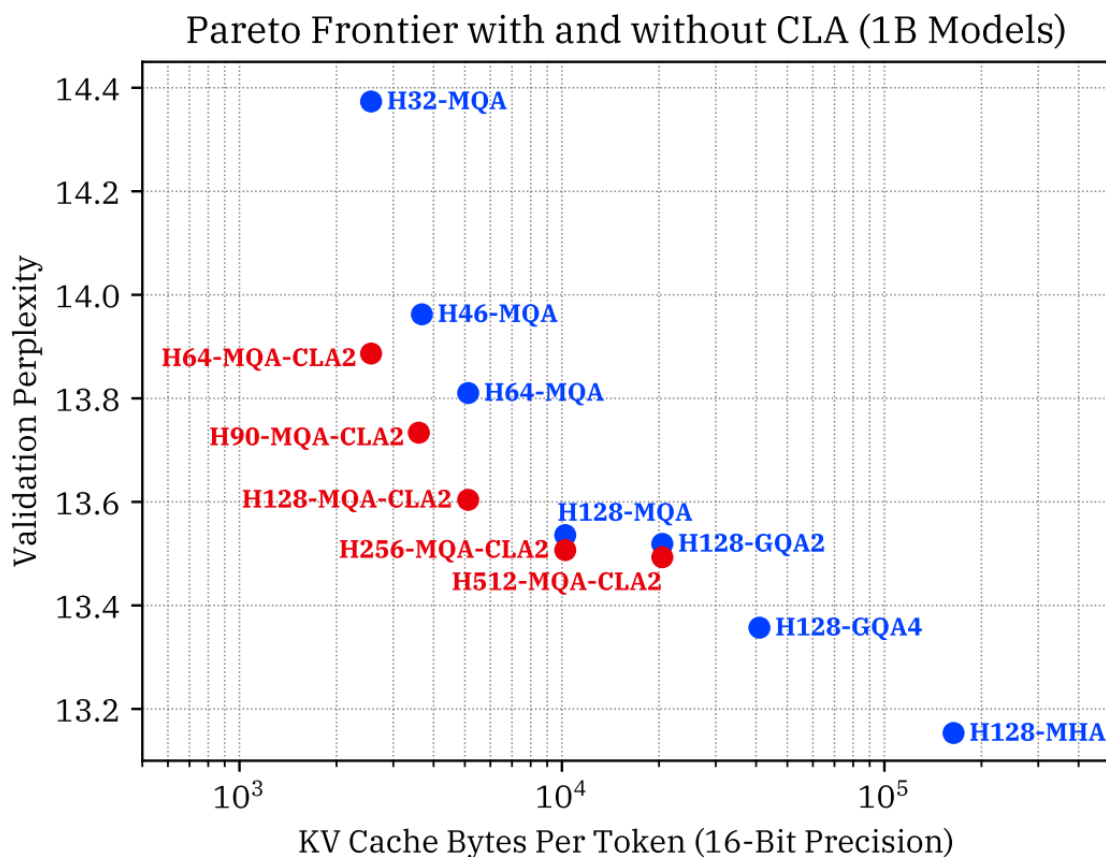
准确结论是：在该论文实验里，MLA 大幅减少 cache，整体质量与 MHA 相当且多项略好；不能外推成 MLA 必然更准。学生问为什么不直接缩小整个模型维度，讲者也明确说缺少对应 ablation，只给出设计直觉：目标化压缩 KV 可能比无差别缩窄整个主干更能保留能力。读者可以把这个直觉与 5.1 的统一表对照：MLA 在“每层每 token 字节数”这一栏上接近 MQA 的水平，却通过 latent 到多头的可学习恢复，保留了远高于 MQA 的表示自由度。

5.4 5.4 CLA：把共享轴从 head 推到 layer

GQA 在 heads 间共享，Cross-Layer Attention 则让若干层共用 K/V。上层 attention 仍会计算自己的 query 和注意力输出，只是不再拥有独立的 K/V 投影与 cache。若每 r 层共享一份，cache 总量就除以 r ；同时参数量也减少（被共享的层不再需要自己的 W_K, W_V ）。



传统结构每层各自产生 K/V ；CLA 的上层通过红色连线复用下层 K/V 。



横轴是每 *token* 的 16-bit KV cache 字节数，纵轴是 *validation perplexity*，越左下越好；红色 CLA2 点把前沿推向更小 *cache*。

图中展示的是 Pareto 改善，不是“所有 CLA 配置都无条件支配所有基线”。共享跨得越多，越可能丢失每层独立表示能力，仍需通过实验选点。这张图也示范了评估 KV 压缩的正确坐标系：横轴用“每 *token* 字节数”这个与硬件直接挂钩的量，纵轴用质量指标，比较的是整条前沿而不是孤立的点。

5.5 Local、hybrid 与 recurrent state：压缩时间轴

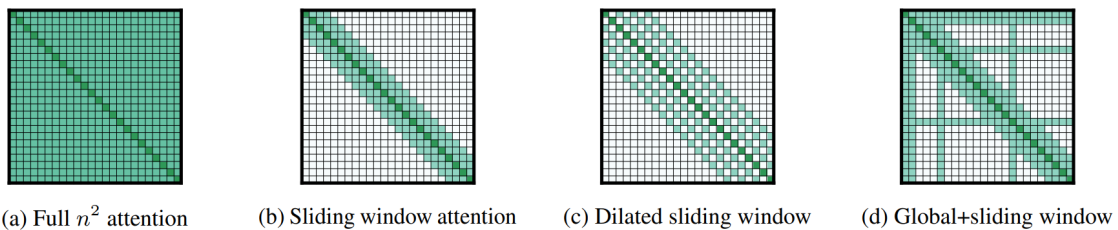
滑动窗口 attention 不再保留全部 S 个历史 *token*，而只保留最近 w 个。其 *cache* 数量级从随总上下文增长变为固定窗口：

$$O(BLSKH) \rightarrow O(BLwKH).$$

- B : batch size;
- L : 层数;
- S : 总上下文长度;
- w : 固定滑动窗口宽度;
- K : KV head 数;
- H : 每 head 维度。

代入 4.2 的模型看收益：decode 每步搬运的 KV 从 $O(BSKH)$ 降到 $O(BwKH)$ ，当 $S \gg w$ 时

这一步的字节数不再随上下文增长，长对话的稳态 decode 延迟从“越来越慢”变成“恒定”。



依次为 *full*、*sliding window*、*dilated sliding window*、*global+sliding window*；稀疏 *attention* 不只有一种固定图案。

多层堆叠能让信息逐层越过单层窗口，有效感受野大致随层数增长——第 ℓ 层的表示最多“看到” ℓw 步之前的信息，这与 CNN 中感受野随深度扩张的机制完全同构；但这不等于远处 token 可被无损随机访问。纯 local attention 会伤害长程检索，所以常把 local 和 global 层交错成 hybrid model。

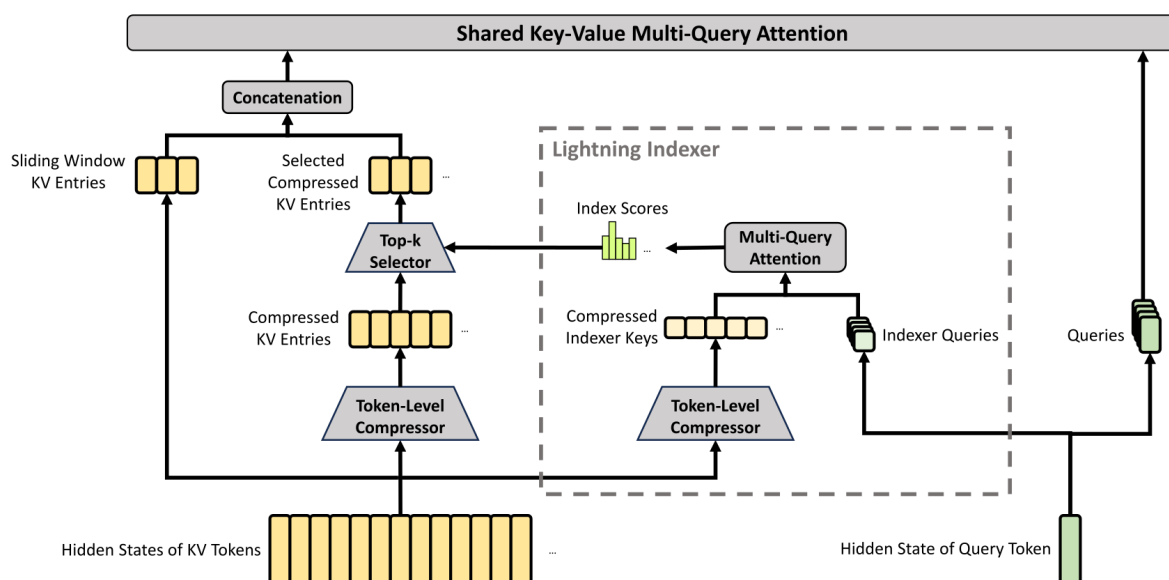
现场问答给出另一个有用的三分法：

- sliding window 保存最近内容的精细细节；
- linear attention、Mamba、DeltaNet 一类固定递归状态保存更久历史的压缩摘要；
- full attention 为远处具体信息提供精确访问。

把整段历史塞进固定小状态必然丢信息，needle-in-a-haystack 任务尤其容易暴露问题。因此“状态与序列长度无关”不等于“记住了全部历史”。用信息论的语言说：长度为 S 的历史包含 $O(S)$ 比特，任何 $O(1)$ 大小的状态都只能保留下一个依赖任务的有损投影；问题只在于丢掉的是否恰好是任务不需要的部分。

5.6 DeepSeek 的组合路线：压缩、索引、Top-k 与局部窗口

课程最后展示 DeepSeek-V4 的组合结构。历史 token 先经 token-level compressor 得到 compressed KV；另一路生成便宜的 indexer keys。当前 query 通过轻量 MQA 产生 index scores，Top-k selector 选出少量压缩条目，再与最近的 sliding-window K/V 拼接，交给顶部共享 KV attention。



图中可直接确认 *compressor*、*lightning indexer*、*Top-k*、*sliding-window entries*、*concatenation* 和 *shared-KV MQA* 的数据流。

字幕可以明确对应：CSA 把每 m 个 token 压成一个表示，DSA 从压缩条目中选 Top-k；讲者对 HCA 只说“进一步压缩”，没有把它严谨映射到图中某一独立模块，因此本文不自行指定。

把这条路线放回 5.1 的地图：它同时动了三个轴——compressor 压历史数量 ($S \rightarrow S/m$)、Top-k 进一步压实际参与读取的条目数、sliding window 保证近处细节，而 shared-KV 则沿 head 轴压缩。它的解码成本结构变成了“便宜的索引打分（全历史）+ 昂贵的精确 attention（少数条目）”，这是稀疏注意力共同的会计方式：用一个粗略的全局扫描，换取精细模块只处理少量候选。

5.7 本章小结

- KV cache 的两个基本问题是“每个 token 存多宽”和“保存多少 token”。
- GQA 压 head 数，MLA 压 latent 维，CLA 跨层共享，local/sparse attention 压历史数量。
- 固定 batch 下减少 cache 可同时改善 latency 与 throughput；释放的内存还能容纳更大 batch。
- 所有压缩都可能损失表达力或增加额外计算，质量和端到端性能必须在具体设置中验证。

6 6. 量化与剪枝：让权重和状态本身更小

6.1 6.1 标量量化先说明了什么

架构压缩减少要保存的元素个数；量化则减少每个元素的字节数。在 memory-bound 区域，低精度同时降低显存容量与 HBM 传输，但必须控制舍入、截断和异常值带来的误差。

最小的非对称量化例子是：

$$x_q = \text{round}\left(\frac{x}{s}\right) + z, \quad \hat{x} = (x_q - z)s.$$

- x ：原始浮点数；

- s : scale;
- z : zero point;
- x_q : 量化后的整数;
- $\hat{x}$: 反量化近似值。

课程令 $x = 5.2342, s = 0.1, z = 4$, 得到 $x_q = 56$, 反量化为 5.2。这段代码只展示舍入误差; 真实系统还要处理整数范围 clipping、按 tensor/channel/group 选择 scale、异常值和高效 kernel。

我们可以把舍入误差做成一个干净的概率模型。设 s 固定、 x/s 的小数部分均匀分布, 则量化噪声 $e = \hat{x} - x$ 近似服从 $[-s/2, s/2]$ 上的均匀分布, 于是:

$$\mathbb{E}[e] = 0, \quad \text{Var}(e) = \frac{s^2}{12}.$$

- e : 单个元素的量化误差;
- $s^2/12$: 宽度为 s 的均匀分布的方差。

若用 b 比特覆盖动态范围 R (例如 $R = x_{\max} - x_{\min}$), 则 $s = R/2^b$, 量化信噪比随位宽指数增长: 对均匀分布于该范围的信号, $\text{SNR} \approx 12 \cdot 2^{2b} / (R^2 / \sigma_x^2)$ 量级, 取对数后就是经验法则**每增加 1 bit, SNR 改善约 6 dB**。反过来, 从 16 bit 砍到 4 bit, 理论上要付出约 $12 \times 6 = 72$ dB 的 SNR——神经网络的权重与激活分布并非均匀 (通常近似钟形、尾部有异常值), 实际损失由分布形状和 scale 的选取粒度共同决定: 按 tensor 取一个 scale, 尾部异常值会把 R 撑大、让绝大多数小数值共用极少几个量化级别; 按 channel 或按 group (如每 128 个元素) 取 scale, 则把 R 局部化, 等效于用更多存储换取更细的 s 。

均匀量化误差演示: 位宽每 +1, SNR 约 +6 dB

```
import torch
```

```
def quantize(x: torch.Tensor, bits: int):
```

```
    qmax = 2 ** (bits - 1) - 1                # 对称 int 范围
    s = x.abs().max() / qmax                   # 按 tensor 取 scale
    x_q = torch.round(x / s).clamp(-qmax - 1, qmax)
    return x_q * s                             # 反量化
```

```
x = torch.randn(100_000) * 0.05                # 类权重分布
```

```
for bits in (8, 4, 3, 2):
```

```
    x_hat = quantize(x, bits)
    snr = 10 * torch.log10((x ** 2).mean() / ((x - x_hat) ** 2).mean())
    print(f"{bits}-bit: SNR = {snr:5.1f} dB")
```

```
# 典型输出: 8-bit ~ 41 dB, 4-bit ~ 17 dB, 3-bit ~ 11 dB, 2-bit ~ 5 dB
```

精度从 bf16 的 2 bytes 可降到 fp8/int8 的 1 byte, int4 甚至只有 0.5 byte。内存减半不保证端到端时间必然减半: 若硬件缺少相应 kernel, 解包、反量化和混合精度开销可能抵消收益。在 memory-bound 框架下, 量化收益的正确读法是 4.2 节模型中的 M_{param} 与 $M_{\text{KV/seq}}$ 同时乘以位宽比例——权重 int4 后, $B = 1$ 的理想单步延迟近似降到原来的 1/4, 这是量化在 decode 阶段格外有效的原因。

6.2 6.2 QAT、PTQ 与 GPTQ

Quantization-aware training (QAT) 在训练前向中模拟 quantize/dequantize，让权重主动适应量化噪声。优点是质量通常更稳，代价是要重新做昂贵训练。其关键工程细节是梯度如何穿过不可微的 round：通常用 straight-through estimator，前向按量化值算、反向把 round 当作恒等映射近似传梯度。

Post-training quantization (PTQ) 在模型训练后进行，通常用少量校准数据为每层、每 tensor 或每 group 估计 scale/zero point，成本低得多。最简单的 PTQ 是 round-to-nearest (RTN)：逐元素四舍五入。6.1 的噪声模型解释了 RTN 的局限——它把每个权重视为独立，完全不考虑这个权重对网络输出的实际影响。

GPTQ 在 PTQ 基础上使用二阶/Hessian 信息；量化一部分权重时，调整尚未量化的权重来补偿输出误差。其骨架可以从一个二次近似看懂。固定校准输入 X ，某层量化前后的输出误差平方为

$$E = \|WX - \hat{W}X\|_F^2 = \sum_j w_j^\top H w_j \text{ 的各行独立子问题, } H = 2XX^\top,$$

- $W, \hat{W}$ ：量化前后的某行权重；
- H ：输入二阶矩（Hessian 近似）；
- w_j ：待处理的单个权重。

把某一坐标 w_j 量化到最近网格点 $\hat{w}_j$ 后，Optimal Brain Quantization 的更新规则让其余未量化坐标吸收误差：

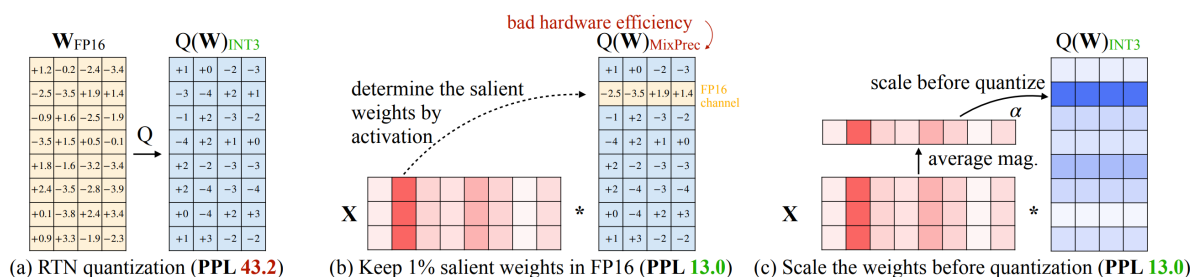
$$\delta = -\frac{w_j - \hat{w}_j}{[H^{-1}]_{jj}} [H^{-1}]_{:,j}, \quad w \leftarrow w + \delta,$$

- δ ：对未量化权重的补偿量；
- $[H^{-1}]_{:,j}$ ：逆 Hessian 的第 j 列。

直觉是：若输入分量之间相关（ H 非对角），减小与 w_j 正相关方向的权重可以抵消 w_j 舍入带来的输出漂移；二阶信息告诉我们要抵消多少。课程只给出机制定位，没有展开完整求解过程，读者只需记住 GPTQ 的误差来源主要是 Hessian 近似与逐层独立处理（误差会沿层累积传播）。

6.3 6.3 AWQ：不要把动机实验当成最终算法

AWQ 的观察是：少量 activation channels 特别大，与这些通道相乘的权重对输出更敏感。最直接的想法是让约 1% 显著权重保持 FP16，其余量化为 INT3；这能显著降低困惑度，却造成不规则混合精度，硬件执行效率差。



左为直接 RTN；中间保留 1% FP16 虽改善 PPL，却被标为 *bad hardware efficiency*；右侧真正的 AWQ 通过 *activation-aware scaling* 后统一量化为 INT3。

AWQ 的关键是按 activation magnitude 对显著通道做缩放，再统一量化低比特权重。代数上，对某个输入通道 j ，把权重列 w_j 乘以 $s_j > 1$ 、同时把该通道的激活除以 s_j ，乘积 $w_j x_j$ 不变：

$$\hat{w}_j = \text{Quant}(s_j w_j), \quad \text{前向时以 } (s_j w_j) \cdot (x_j / s_j) \text{ 的等价形式计算.}$$

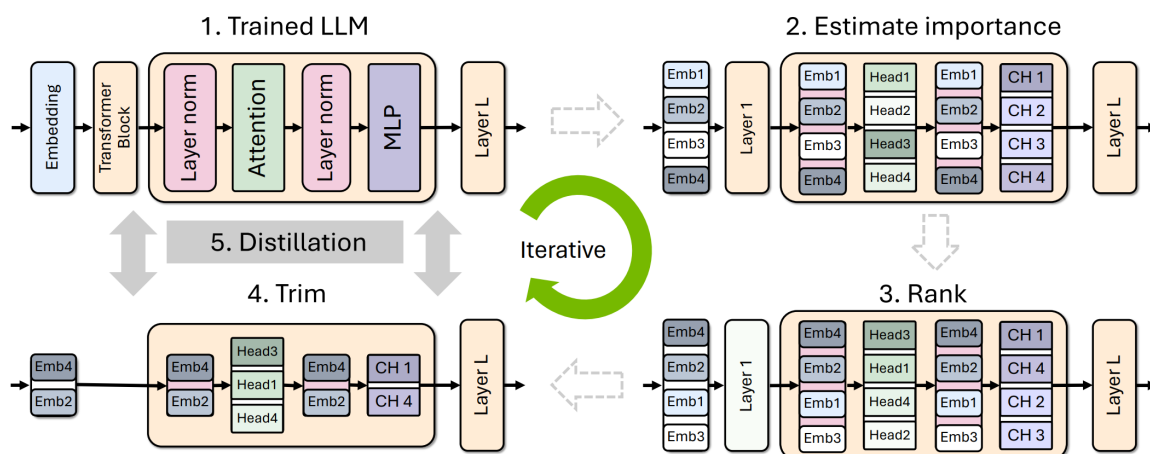
由 6.1 的噪声模型，RTN 的相对误差约为 $s_{\text{grid}} / (2|w|)$ 量级；把权重先放大 s_j 倍再量化，量化网格相对原权重等效变细了 s_j 倍，显著通道的绝对误差被压小，而激活侧除以 s_j 只是把大激活变小，不会引入新的 clipping 风险（大激活本来就在范围内，缩小更安全）。 s_j 按校准集上的平均激活幅度选取，这就是“activation-aware”的含义。

图中 RTN 的 PPL 为 43.2，混合精度和 scale-before-quantize 两种方案都到 13.0，但后者保留规则 INT3 布局。课程报告 fp16→int3 约 $4\times$ 更低内存、 $3.2\times$ 加速；这是特定论文实验，不是所有模型和硬件的固定保证——加速比取决于 kernel 是否真能用 int3 权重做高效反量化矩阵乘，内存比例则是位宽比的直接结果（ $16/3 \approx 5.3$ ，计入 scale/zero point 元数据后约 $4\times$ ）。

注意边界 讲者口述把 AWQ 简化成“保留 1% 高精度权重”，但官方图明确把它标成硬件效率差的中间方案。最终算法的教学重点必须放在 activation-aware scaling 后的统一低比特量化。

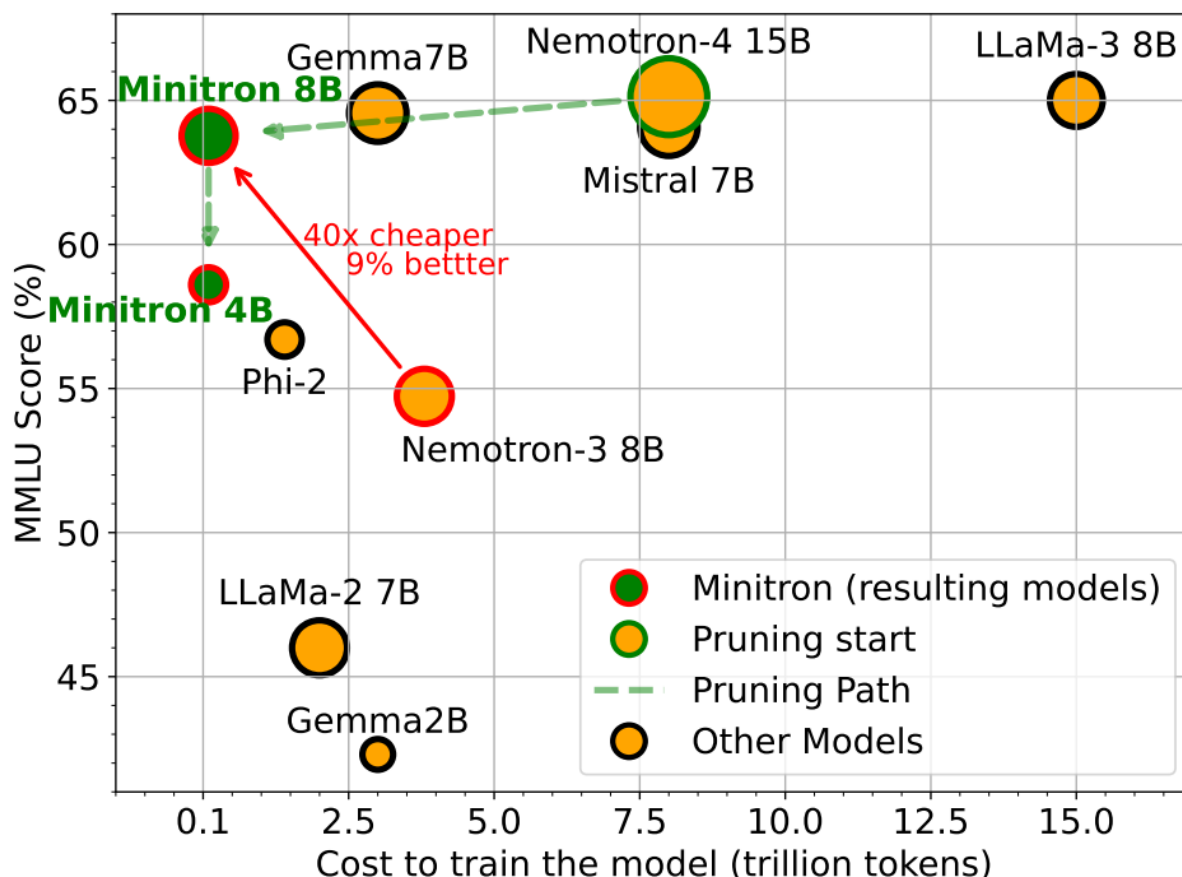
6.4 结构化剪枝：删掉组件，再用蒸馏修复

剪枝的思路更直接：估计 layer、attention head、embedding/hidden dimension、MLP channel 的重要性，删除不重要结构，再让原模型充当 teacher 修复小模型。



完整流程是 *trained LLM* $\rightarrow$ *estimate importance* $\rightarrow$ *rank* $\rightarrow$ *trim* $\rightarrow$ *distillation*，并可迭代执行。

校准集只需约 1024 个样本，但“重要性”不是一条永恒的幅值定理。激活均值、方差、校准分布、剪后验证和各组件相互作用都可能影响排序。删掉 layer/head/channel 得到的模型结构发生了真实变化，必须蒸馏修复，而不是指望剩余权重自动组成好模型。从性能模型的角度看，结构化剪枝与量化的差别在于：量化把 M_{param} 乘上一个位宽比例，剪枝则直接减小 D 、 L 、 F 等结构常数——后者同时减少 FLOPs 和字节数，因此即便进入 compute-bound 区也依然有效，但结构改变对质量的扰动也更剧烈，这就是蒸馏修复不可省略的原因。



横轴是训练模型所用 *token* 成本，纵轴是 *MMLU*；图中的“ $40\times$ cheaper, 9% better”比较的是得到小模型的训练路线，不是宣称单 *token* 推理 $40\times$ 。

课程把实践配方分为两类：

- 从头训练：先定义更快架构，再直接训练它；
- 蒸馏修复：定义更快架构，从原模型中选取或拼接权重初始化，再通过 *distillation repair*。

后者充分利用昂贵大模型已经学到的知识，但剪枝所得“Frankenstein”初始化不是成品，修复阶段不可省略。Minitron 式结果之所以成立，正是因为蒸馏让修复所需的 *token* 数远小于从头训练同一架构——这张图的横轴读法必须是“训练预算”，否则会把训练侧结论误当成推理侧结论。

6.5 本章小结

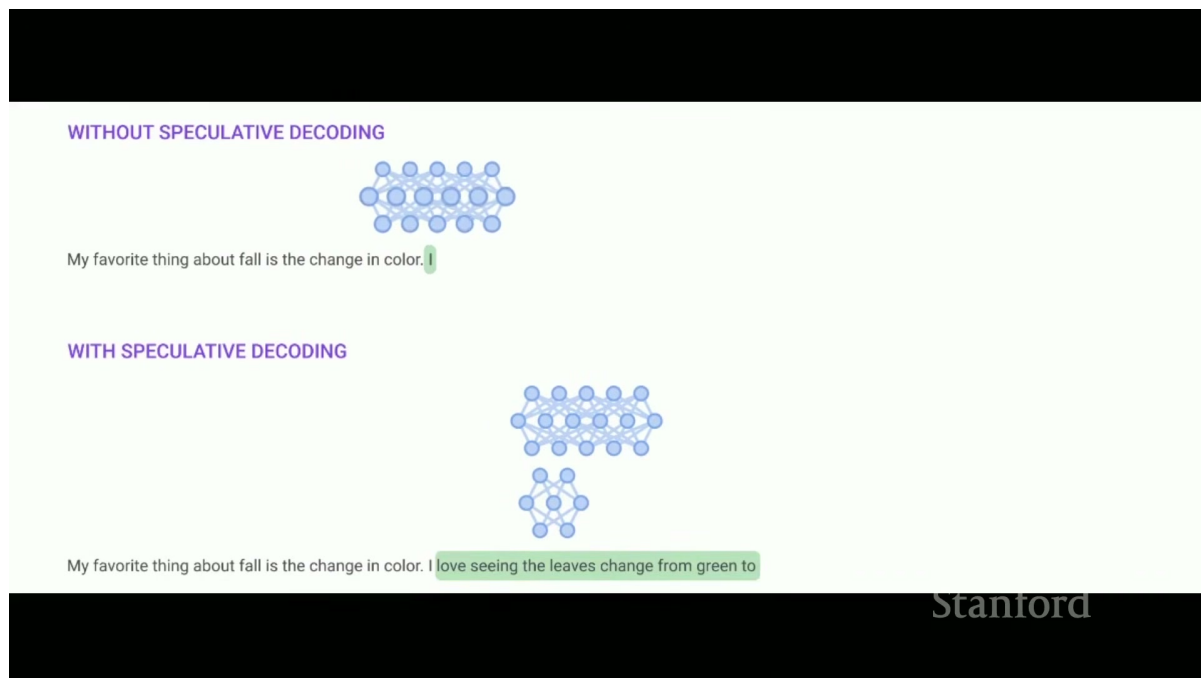
- 量化减少每个数的字节数，剪枝减少真实存在的参数与结构。
- QAT 用重新训练换质量，PTQ 用校准降低成本，GPTQ 用二阶信息补偿量化误差。
- AWQ 的最终机制是 activation-aware scaling 后统一低比特量化，而不是永久保留不规则 1% FP16。
- 结构化剪枝必须与重要性校准和蒸馏修复配套；结果图要区分训练成本与推理成本。

7 7. 投机采样：小模型先猜，大模型并行验证

7.1 7.1 为什么“检查”可能比“生成”快

逐 token decode 要把大模型权重和 KV cache 一步一步搬进来；但给定一串候选 token 后，target model 可以像 prefill 一样，并行计算多个位置的 logits。于是出现一个不对称：**用大模型串行生成很慢，用大模型并行检查一串候选却相对便宜**。用第 2、4 节的语言重述：串行生成 K 个 token 需要 K 次完整的前向，每次都把全部权重从 HBM 搬一遍；而把 K 个候选拼成 $T = K$ 的一次前向，权重仍然只搬一遍——在 memory-bound 区域，后者的墙钟时间几乎与 $T = 1$ 相同，因为时间由字节数而非 FLOPs 决定。

投机采样用小型 draft model p 连续猜 K 个 token，再让 target model q 一次并行检查这些位置。若 draft 足够便宜、又与 target 足够接近，一次昂贵 target 前向可以接受多个 token，使输出以“burst”形式前进。



上方 target 每次只前进一个 token；下方小 draft model 先提出一串候选，再由大 target model 一次验证后成批接受。

核心提示 Draft 不是最后的决策模型。它只负责提出候选；target 的概率与接受—补偿规则共同保证最终仍采自 target 分布。

7.2 接受、拒绝与残差补偿

在某个已经接受的前缀下，draft 提议 token x 。它被接受的概率是：

$$a(x) = \min \left(1, \frac{q(x)}{p(x)} \right).$$

- $a(x)$ ：候选 token 的接受概率；
- $p(x)$ ：draft 在当前前缀下给 x 的概率；
- $q(x)$ ：target 在同一前缀下给 x 的概率。

若 $q(x) \geq p(x)$ ，draft 没有过度提议这个 token，候选必然接受；若 $q(x) < p(x)$ ，只按比例 q/p 接受，抵消 draft 的过采。这个比值的含义可以对照拒绝采样理解：我们把 p 当作提议分布、把 q 当作目标分布， $a(x)$ 正是经典接受—拒绝采样中“以 $q(x)/(Mp(x))$ 接受”的规则在 $M = 1$ 时的形式；不同的是这里允许 $q > p$ 的 token 必然通过，因此整体接受率远高于传统拒绝采样。

一旦候选被拒绝，就不能简单从原始 q 再采一次，否则会重复计算已经由接受分支覆盖的概率质量。正确的修正分布是正残差的归一化：

$$r(x) = \frac{\max(q(x) - p(x), 0)}{\sum_y \max(q(y) - p(y), 0)}.$$

- $r(x)$ ：拒绝后用于采样修正 token 的概率；
- $q(x) - p(x)$ ：target 相对 draft 尚未覆盖的概率质量；
- y ：对整个词表求和的索引。

首次拒绝后，本轮剩余 draft tokens 都是在已经失效的条件前缀上生成的，因此必须丢弃并结束本轮。若 K 个候选全部接受，target 已经顺便算出了第 $K + 1$ 个位置的分布，可再从 q 采一个额外 token。也就是说，无论接受多少，每一轮至少前进一个 token——这正是“投机采样永远不会比逐 token decode 走得更慢（在 token 数意义上）”的保证。

Algorithm 2 Speculative Sampling (SpS) with Auto-Regressive Target and Draft Models

Given lookahead K and minimum target sequence length T .

Given auto-regressive target model $q(\cdot|\cdot)$, and auto-regressive draft model $p(\cdot|\cdot)$, initial prompt sequence $x_0, \dots, x_t$.

Initialise $n \leftarrow t$.

while $n < T$ **do**

for $t = 1 : K$ **do**

 Sample draft auto-regressively $\tilde{x}_t \sim p(x|x_1, \dots, x_n, \tilde{x}_1, \dots, \tilde{x}_{t-1})$

end for

 In parallel, compute $K + 1$ sets of logits from drafts $\tilde{x}_1, \dots, \tilde{x}_K$:

$$q(x|x_1, \dots, x_n), q(x|x_1, \dots, x_n, \tilde{x}_1), \dots, q(x|x_1, \dots, x_n, \tilde{x}_1, \dots, \tilde{x}_K)$$

for $t = 1 : K$ **do**

 Sample $r \sim U[0, 1]$ from a uniform distribution.

if $r < \min\left(1, \frac{q(x|x_1, \dots, x_{n+t-1})}{p(x|x_1, \dots, x_{n+t-1})}\right)$ **then**

 Set $x_{n+t} \leftarrow \tilde{x}_t$ and $n \leftarrow n + 1$.

else

 sample $x_{n+t} \sim (q(x|x_1, \dots, x_{n+t-1}) - p(x|x_1, \dots, x_{n+t-1}))_+$ and exit for loop.

end if

end for

 If all tokens $x_{n+1}, \dots, x_{n+K}$ are accepted, sample extra token $x_{n+K+1} \sim q(x|x_1, \dots, x_n, x_{n+K})$ and set $n \leftarrow n + 1$.

end while

论文伪代码包含 $K + 1$ 组 *target logits*、接受概率 $\min(1, q/p)$ 、拒绝后的正残差，以及全接受后的额外 *token*。

7.3 用二元词表证明为什么它是 exact sampling

视频因时间略过了证明，但这是理解残差分布不可删除的关键。设词表只有 $\{A, B\}$ ，draft 对 A 过采，即 $p(A) > q(A)$ ，于是 $p(B) < q(B)$ 。

A 只有一条输出路径：draft 采到 A 且通过接受检验。因此：

$$P(\text{输出}A) = p(A) \frac{q(A)}{p(A)} = q(A).$$

- $P(\text{输出}A)$ ：投机采样最终输出 A 的概率；
- $p(A)$ ：draft 提议 A 的概率；
- $q(A)/p(A)$ ：A 在过采情况下被接受的概率；
- $q(A)$ ：target 对 A 的目标概率。

B 有两条路径：draft 直接采到 B，此时必然接受；或者 draft 采到 A 但被拒绝，残差概率全部补给 B。两条相加：

$$\begin{aligned} P(\text{输出}B) &= p(B) + p(A) \left(1 - \frac{q(A)}{p(A)}\right) \\ &= p(B) + p(A) - q(A) \\ &= q(B). \end{aligned}$$

- $P(\text{输出}B)$: 投机采样最终输出 B 的概率;
- $p(B)$: draft 直接提出 B 的概率;
- $p(A)(1 - q(A)/p(A))$: draft 提出 A 但拒绝后补偿为 B 的概率;
- 最后一行使用 $p(A) + p(B) = q(A) + q(B) = 1$ 。

一般词表中，论证结构完全相同。先算拒绝事件的总概率：

$$\beta = \sum_y p(y) \left(1 - \min \left(1, \frac{q(y)}{p(y)} \right) \right) = \sum_y \max(p(y) - q(y), 0).$$

- β : 一个候选被拒绝的总概率;
- 第二行把 $p(y) - \min(p(y), q(y))$ 改写为 $\max(p(y) - q(y), 0)$ 。

注意一个关键恒等式：因为 $\sum_y (p(y) - q(y)) = 0$ ，正部与负部的总量相等，即

$$\sum_y \max(p(y) - q(y), 0) = \sum_y \max(q(y) - p(y), 0),$$

所以残差分布 r 的归一化常数恰好也是 β 。于是任意 token x 的总输出概率为：

$$\begin{aligned} P(\text{输出}x) &= \underbrace{p(x) \min \left(1, \frac{q(x)}{p(x)} \right)}_{\text{接受分支}} + \underbrace{\beta \cdot r(x)}_{\text{拒绝后补偿}} \\ &= \min(p(x), q(x)) + \beta \cdot \frac{\max(q(x) - p(x), 0)}{\beta} \\ &= \min(p(x), q(x)) + \max(q(x) - p(x), 0) \\ &= q(x). \end{aligned}$$

最后一步是恒等式 $\min(a, b) + \max(b - a, 0) = b$ ：当 $q \geq p$ 时第一项为 p 、第二项为 $q - p$ ；当 $q < p$ 时第一项为 q 、第二项为零。接受分支对 token x 提供 $\min(p(x), q(x))$ ，残差分支再提供 $\max(q(x) - p(x), 0)$ ，两者和恰好是 $q(x)$ 。逐位置在条件前缀下应用同样论证，由条件概率的链式法则，就得到与 target 自回归采样相同的联合分布。

注意边界 “Exact” 指在相同 logits 处理和采样设定下，输出分布精确等于 target；它不意味着两次随机运行会逐 token 相同，也不意味着任意近似 speculative decoding 实现都自动保持分布。常见的破坏 exactness 的近似包括：省略残差重采样、接受后直接续采剩余候选、以及在 top-p/top-k 截断不一致的两个模型间做投机。

期望接受率与加速比的估算模型 定义单 token 的期望接受率 α 为 “draft 提议的 token 被接受的概率”：

$$\alpha = \sum_x p(x) \min \left(1, \frac{q(x)}{p(x)} \right) = \sum_x \min(p(x), q(x)) = 1 - \frac{1}{2} \sum_x |p(x) - q(x)|,$$

- α : 期望接受率;

- $\frac{1}{2} \sum_x |p(x) - q(x)|$: p 与 q 的总变差距离 (total variation distance)。

第三步用到恒等式 $\min(p, q) = \frac{p+q-|p-q|}{2}$ 与 $\sum p = \sum q = 1$ 。这个等式把工程目标说得很清楚：**接受率完全由两个分布的 TV 距离决定**，draft 与 target 越接近， α 越接近 1。

在“各候选独立、接受率恒为 α ”的简化假设下，一轮提议 K 个 token，第 i 个候选被走到且接受的概率为 α^i ，再加上每轮必有的一个修正/额外 token，每轮期望前进的 token 数为：

$$\mathbb{E}[\text{tokens/round}] = \sum_{i=1}^K \alpha^i + 1 \cdot \alpha^K + \beta' \cdot 1 = \frac{1 - \alpha^{K+1}}{1 - \alpha},$$

- 等比级数求和： $1 + \alpha + \dots + \alpha^K = \frac{1 - \alpha^{K+1}}{1 - \alpha}$ ；
- 直观解读：第 0 个 token（修正或额外 token）必得，之后每多走一个候选都要再乘一次 α 。

再看成本。设 draft 单步成本为 t_d ，target 一次前向成本为 t_q （验证 $K+1$ 个位置在 memory-bound 区域约等于一次单步前向），则一轮墙钟时间约为 $Kt_d + t_q$ ，而朴素 decode 产出同样多 token 需要 $\mathbb{E}[\text{tokens}] \cdot t_q$ 。加速比的估算模型为：

$$\text{speedup} \approx \frac{\mathbb{E}[\text{tokens/round}] \cdot t_q}{K t_d + t_q} = \frac{(1 - \alpha^{K+1})/(1 - \alpha)}{1 + K \cdot t_d/t_q}.$$

- 分子：每轮期望产出折算成的朴素 decode 时间；
- 分母：每轮实际耗时，含 K 次 draft 前向与一次 target 验证。

代入一组 plausible 数字： $\alpha = 0.8$ ， $K = 4$ ， $t_d/t_q = 0.1$ 。期望 token 数为 $(1 - 0.8^5)/0.2 = (1 - 0.328)/0.2 \approx 3.36$ ，加速比为 $3.36/(1 + 0.4) \approx 2.4\times$ 。若 α 降到 0.5，同式给出 $(1 - 0.5^5)/0.5/1.4 \approx 1.38\times$ ——接受率对收益的影响是指数级的，这就是 draft 质量比 draft 速度更值得投入的原因。

投机采样单步的玩具实现：draft 提议 + target 并行验证 + 残差修正

import torch

```
def speculative_round(p_logits, q_logits_all, K):
    # p_logits: draft 在 K 个位置上的 logits, shape [K, V]
    # q_logits_all: target 并行验证得到 K+1 个位置的 logits, shape [K+1, V]
    p = torch.softmax(p_logits, dim=-1)
    q = torch.softmax(q_logits_all, dim=-1)
    accepted = []
    for i in range(K):
        x = torch.multinomial(p[i], 1).item()          # draft 提议第 i 个 token
        accept_prob = min(1.0, (q[i, x] / p[i, x]).item())
        if torch.rand(()) < accept_prob:
            accepted.append(x)                          # 接受，继续验证下一个
        else:
            r = (q[i] - p[i]).clamp(min=0)              # 正残差
```

```

r = r / r.sum() # 归一化为修正分布
accepted.append(torch.multinomial(r, 1).item())
return accepted # 首次拒绝即结束本轮
bonus = torch.multinomial(q[K], 1).item() # 全接受：额外采一个
return accepted + [bonus]

```

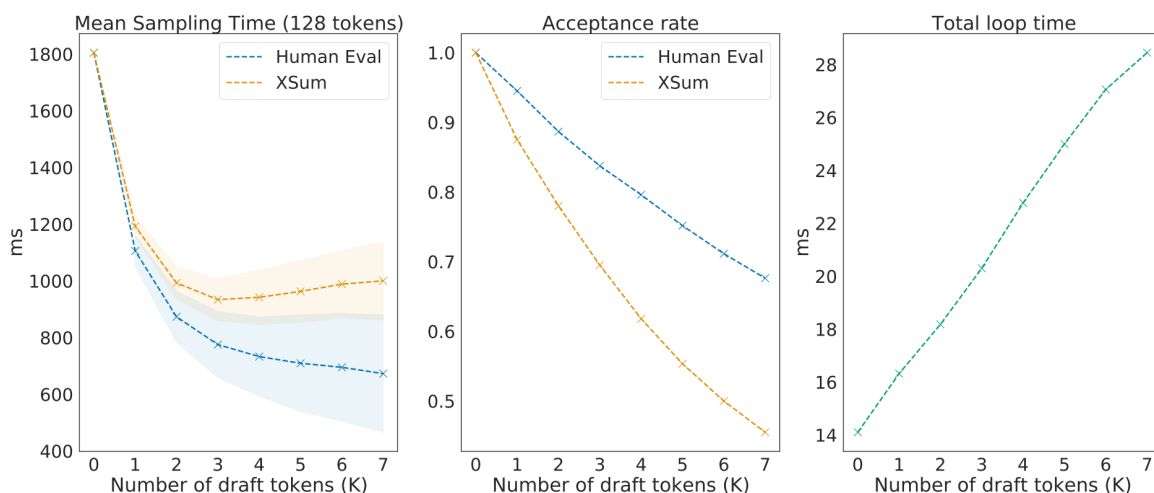
7.4 7.4 K 的甜点区取决于任务、模型和硬件

Table 1 | **Chinchilla performance and speed on XSum and HumanEval with naive and speculative sampling at batch size 1 and $K = 4$.** XSum was executed with nucleus parameter $p = 0.8$, and HumanEval with $p = 0.95$ and temperature 0.8.

Sampling Method	Benchmark	Result	Mean Token Time	Speed Up
ArS (Nucleus)	XSum (ROUGE-2)	0.112	14.1ms/Token	1×
SpS (Nucleus)		0.114	7.52ms/Token	1.92×
ArS (Greedy)	XSum (ROUGE-2)	0.157	14.1ms/Token	1×
SpS (Greedy)		0.156	7.00ms/Token	2.01×
ArS (Nucleus)	HumanEval (100 Shot)	45.1%	14.1ms/Token	1×
SpS (Nucleus)		47.0%	5.73ms/Token	2.46×

在 $batch\ 1$ 、 $K = 4$ 的 *Chinchilla* 实验里，*XSum* 约 $1.92\times/2.01\times$ ，*HumanEval* 约 $2.46\times$ ，任务指标基本持平或小幅波动。

不能据此直接承诺所有部署都有 $2\times$ 。收益由四个量共同决定：draft 的单步成本、draft 与 target 的一致性、target 并行验证 $K + 1$ 个位置的成本，以及接受后平均能前进多少 token。把上面的加速比模型对 K 求最优：分子随 K 增大趋于饱和值 $1/(1 - \alpha)$ ($\alpha < 1$ 时 $\alpha^{K+1} \rightarrow 0$)，分母却随 K 线性增长，因此必存在有限的甜点 K^* ； α 越高，饱和越晚、 K^* 越大。



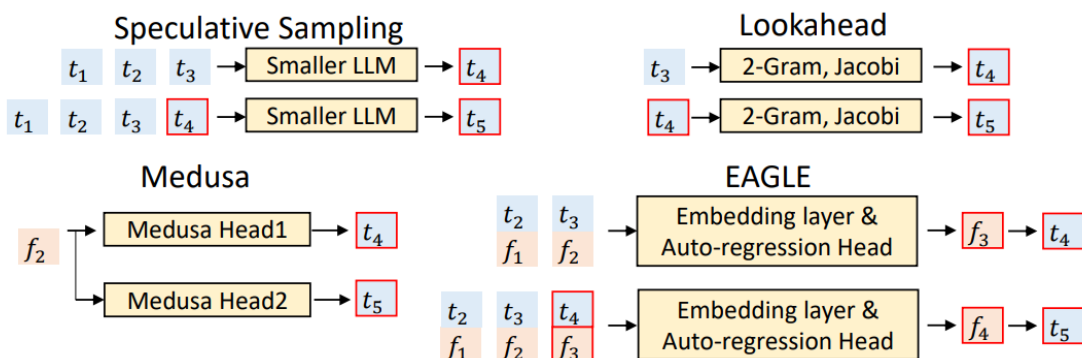
K 增大时 *target* 每轮验证成本上升、接受率下降；*XSum* 在图中约 $K = 3$ 最低，*HumanEval* 到 $K = 7$ 仍下降，说明甜点区任务相关。

图中还隐含一个更细的事实：接受率本身随候选位置衰减——第 1 个猜测只要求分布接近，第 i 个猜测要求 draft 连猜对 $i - 1$ 步后的条件分布依然接近，因此越靠后的候选越“不值钱”，加大 K 的

边际收益递减。代码生成 (HumanEval) 的接受率高于摘要 (XSum)，因为代码的局部确定性更强，draft 更容易猜中，这解释了两者的甜点位置的差异。

课程给出的常见尺度是 70B target 配 8B draft，或 8B target 配 1B draft。通过蒸馏让 draft 更接近 target 可以提高接受率，但训练和部署 draft 本身也有成本。

7.5 Medusa 与 EAGLE：改进 draft 的方式



Medusa 用多个 heads 并行提出未来候选；EAGLE 让 draft 利用 target 的高层特征，使候选更贴近 target。

课程仅快速点名这两条扩展路线，没有展开训练目标和候选树验证细节。它们最适合作为“draft 仍有巨大设计空间”的拓展，而不是冒充本课已经完整讲解的算法。用 7.4 的模型看，两者都在优化加速比公式的不同因子：Medusa 消掉独立 draft 模型的串行前向（降低 Kt_d ），EAGLE 通过共享 target 特征提高候选质量（提高 α ）。

7.6 本章小结

- 投机采样利用了“串行生成慢、并行检查快”的不对称。
- 接受概率纠正 draft 的过采，正残差分布补回 target 少采的概率质量。
- 正确算法保持 target 分布不变；省略残差或错误处理拒绝位置就不再 exact。
- K 越大并不必然越快，最优值由接受率、draft 成本和 target 验证成本共同决定。

8 8. 动态工作负载：Continuous Batching 与 PagedAttention

8.1 8.1 在线请求为什么不是训练中的整齐矩形

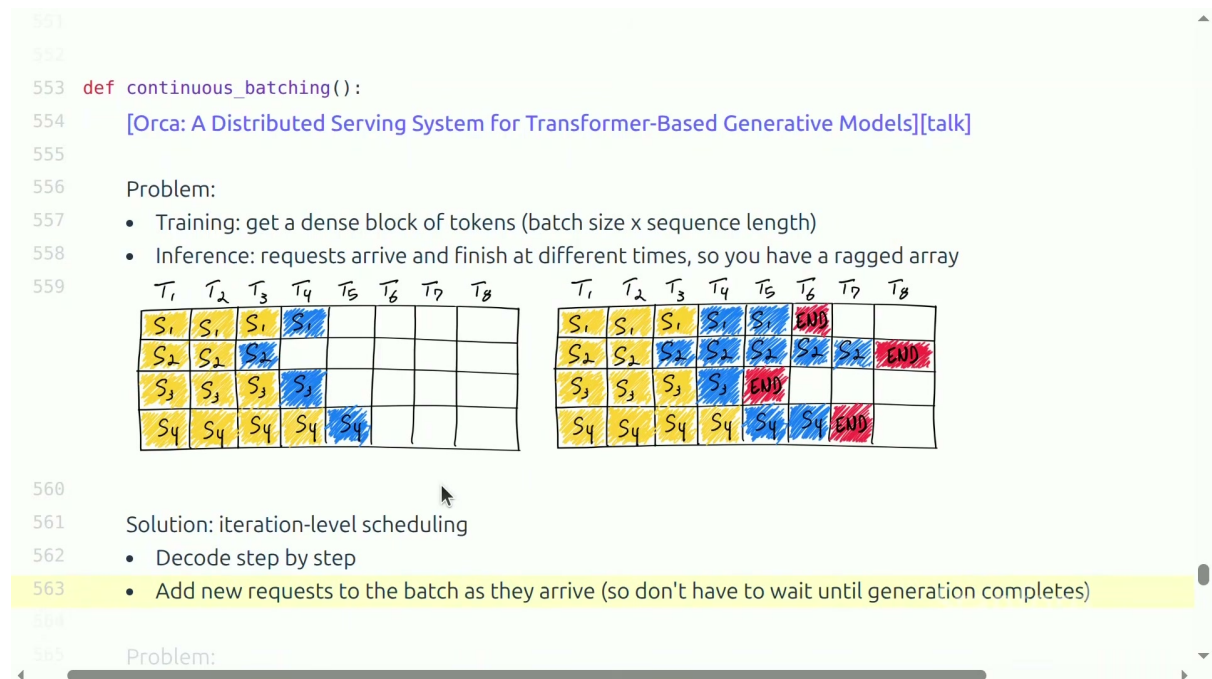
线上流量具有三种动态性：请求在不同时间到达，prompt 和输出长度不同，许多请求又可能共享 system prompt 或 few-shot 前缀。若使用静态 batch，一条长响应会让已经结束的槽位空等，新请求也可能必须等整批完成，TTFT 和利用率都受损。把第 4 节的模型套上来：静态 batch 的吞吐公式 $\text{throughput} = B/\ell$ 假设所有 B 个槽位始终活跃，而真实负载下有效 batch 随时间衰减——一条 2000 token 的请求和十条 50 token 的请求同批，后五十步里有效 B 只剩 1，带宽被白白读权重却几乎不产出 token。

8.2 Continuous batching: 每个 decode step 都重新组织 batch

Orca 的 iteration-level scheduling 把调度粒度从“整条请求生成完”降到“每个 decode iteration”:

1. 当前 batch 中每条活动序列各生成一个 token;
2. 完成或输出 EOS 的请求立即退出;
3. 新请求进入空出的槽位;
4. 下一轮对更新后的活动集合继续 decode。

因此 batch 不是一群固定成员，而是一条随时间变化的请求流。它解决的是调度与硬件利用率，并没有减少单条序列逻辑上需要的 KV。



左侧静态 batch 在部分序列结束后留下空槽，右侧按 iteration 调度并把新请求接入。

吞吐—延迟权衡的定量读法 Continuous batching 的价值可以用 4.2 的模型精确表述。设系统通过不断补位把活动 batch 维持在 B 附近，则稳态吞吐与单步延迟仍满足：

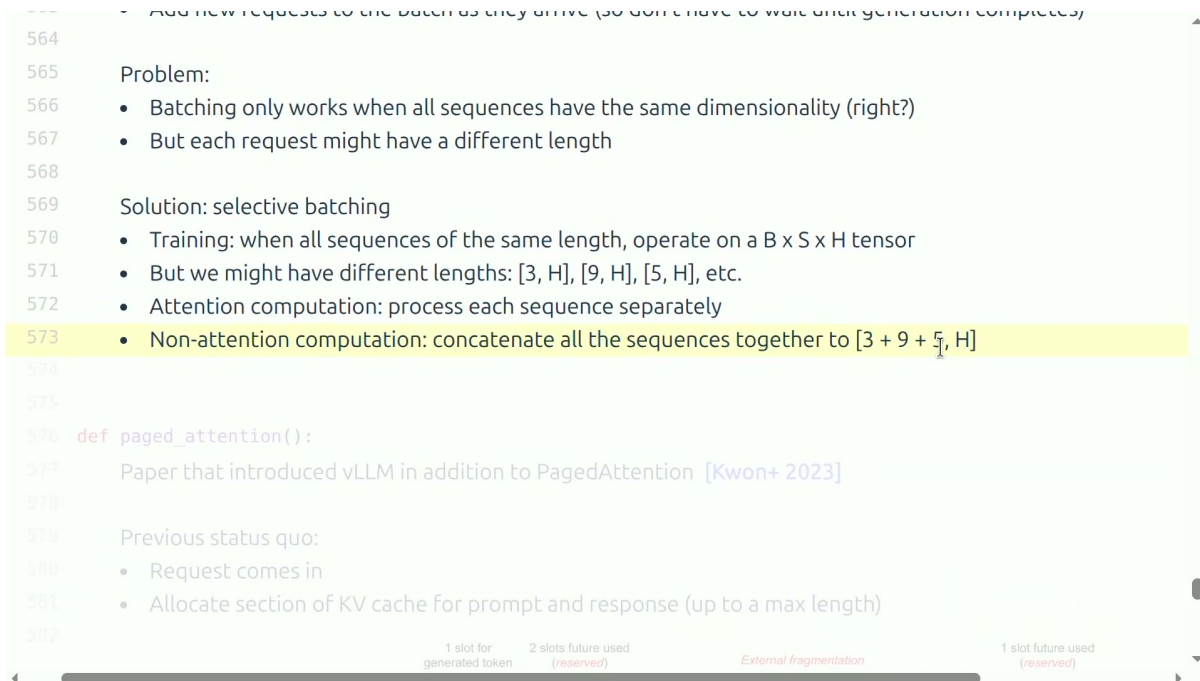
$$\text{throughput} \approx \frac{B \cdot W}{M_{\text{param}} + B \cdot M_{\text{KV/seq}}}, \quad \ell(B) = \frac{M_{\text{param}} + B \cdot M_{\text{KV/seq}}}{W}.$$

- 吞吐随 B 单调上升，但渐近饱和于 $W/M_{\text{KV/seq}}$;
- 单步延迟随 B 线性上升，从 M_{param}/W 起，斜率为 $M_{\text{KV/seq}}/W$ 。

调度器实际做的是在这条权衡曲线上选点：交互服务把 B 压在延迟预算内，离线批处理把 B 推到显存允许的上限 B_{max} (4.3 节对 GQA 13B 算出约 341)。与静态 batch 相比，continuous batching 不改变这条曲线本身，它改变的是系统停留在曲线上的时间比例——空槽被消灭后，有效 B 始终接近目标值，于是平均吞吐逼近曲线的理论值。代价有二：其一，新请求插入需要先完成自己的 prefill，prefill 与 decode 混跑会拉长当轮迭代时间；其二， B 越大，每条请求的 token latency 越高，这就是为吞吐支付的延迟税。

8.3 Selective batching: 不同算子采用不同拼法

长度为 3、9、5 的 ragged sequences 不能无 padding 地堆成统一的 $B \times S \times H$ 张量。Attention 依赖每条序列自己的长度、mask 和 KV cache，需要分别或用 ragged kernel 处理；LayerNorm、线性层和 MLP 等逐 token 运算则不依赖序列边界，可以把 $[3, H]$ 、 $[9, H]$ 、 $[5, H]$ 拼成 $[17, H]$ ，形成更大的矩阵运算。用 3.4 节的结论看，这一步把逐 token 算子的有效行数从“每条序列分开算”提升到“全部活跃 token 一起算”，等效于增大了摊薄权重的 BT 。



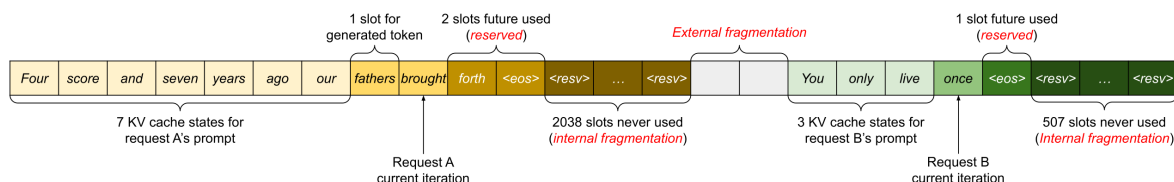
不同长度序列的 *attention* 分开处理，非 *attention* 计算拼成 $[3+9+5, H]$ 。

补充说明 讲者用 3×3 、 9×9 attention 说明长度依赖。带 KV cache 的实际 decode 通常是一个新 query 对各自 S_i 个历史 KV 做 $1 \times S_i$ 交互；课程图的作用是解释 ragged shape，不应被误写成每步重算完整方阵。

8.4 连续预留为什么产生碎片

旧式 KV 管理会在请求到达时，为“prompt + 最大可能输出”预留一段连续空间。输出长度事先未知，于是产生：

- **内部碎片**：请求自己的预留区间里，有大量 future slots 最终从未使用；
- **外部碎片**：不同连续分配区间之间出现小空洞，虽然总空闲容量不少，却无法res满足新的大块连续请求。

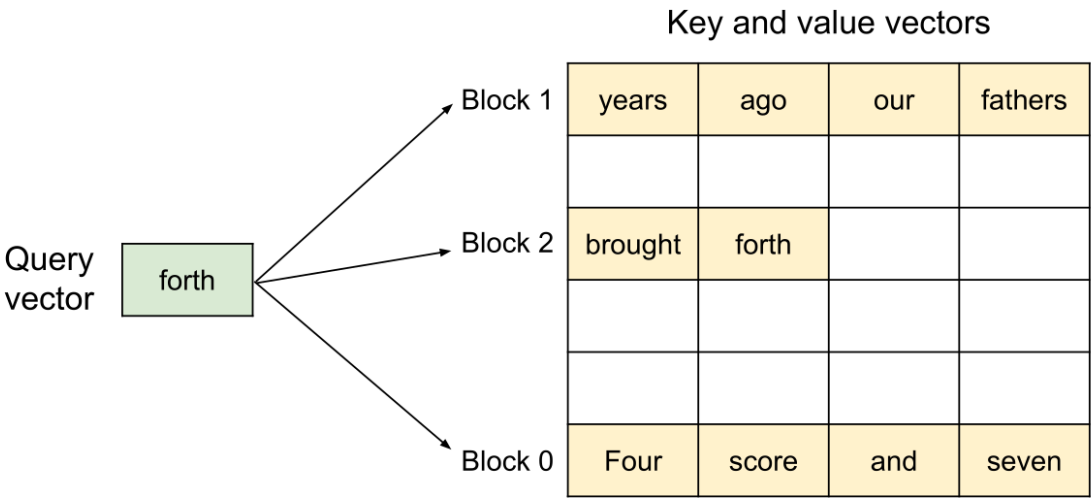


Request A/B 分别留下 2038、507 个 *never-used slots*，请求之间还存在 *external fragmentation*。

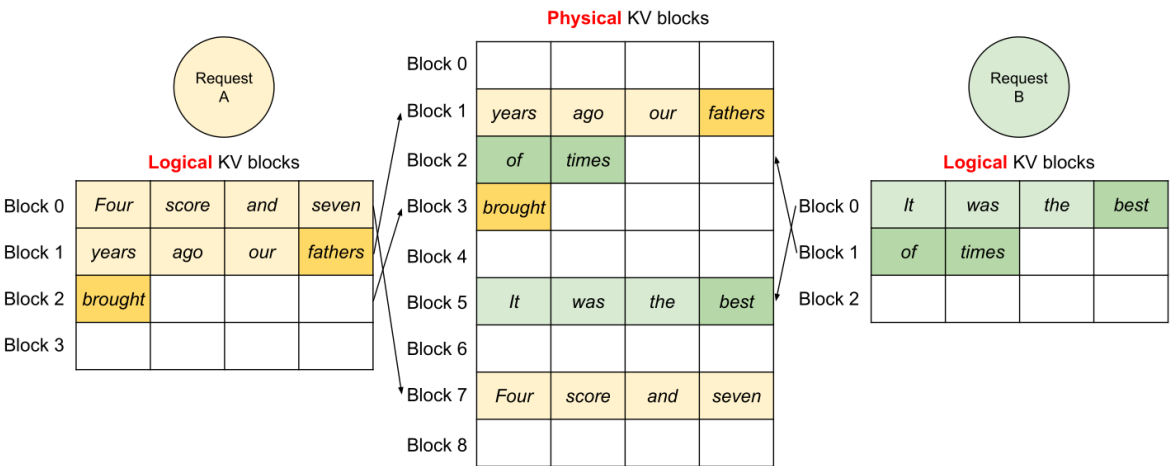
把图中数字放回 4.1 的会计：对 Llama 2 13B，每个 slot 每层要存 $KH \cdot 2 \cdot 2$ 字节（MHA 为 $40 \times 128 \times 4 = 20.5$ KB），全模型合计每 token 约 0.82 MB。Request A 的 2038 个未使用 slot 对应约 $2038 \times 0.82 \approx 1.7$ GB 的死显存——接近 7 条完整 1024-token 序列的有效 KV 预算。碎片在这里不是抽象的操作系统概念，而是直接折算成本可同时服务的请求数。

8.5 8.5 Logical blocks 映射到非连续 physical blocks

PagedAttention 借用了操作系统分页的思想：把一条逻辑序列的 KV 切成固定大小 logical blocks，再由 block table 映射到任意空闲 physical KV blocks。只有当前块填满时才分配下一块，无需为最大输出长度预留连续区间。



query *forth* 需要从 Block 0、1、2 中取得历史 *key/value*；块化不会改变逻辑 *attention*。



Request A/B 的逻辑顺序保持连续，但物理块可以落在 Block 1/2/3/5/7 等不相邻位置。

地址翻译过程与 OS 分页完全同构：序列内第 i 个 token 位于 logical block $\lfloor i/b \rfloor$ 的第 $i \bmod b$ 个槽位（ b 为块内 token 数），block table 把 logical block 号查成 physical block 号，attention kernel 据此聚集出全部历史 K/V。

分页后的碎片分析也变得干净。外部碎片被彻底消除——任何空闲物理块都可服务任何请求；内部碎片只剩每条序列最后一个未填满的块。设块大小为 b 个 token，序列长度对 b 取模后均匀分布，则最后一块的浪费槽位数在 0 到 $b - 1$ 上均匀，期望浪费为：

$$\mathbb{E}[\text{浪费 tokens/seq}] = \frac{0 + 1 + \dots + (b - 1)}{b} = \frac{b - 1}{2} \approx \frac{b}{2}.$$

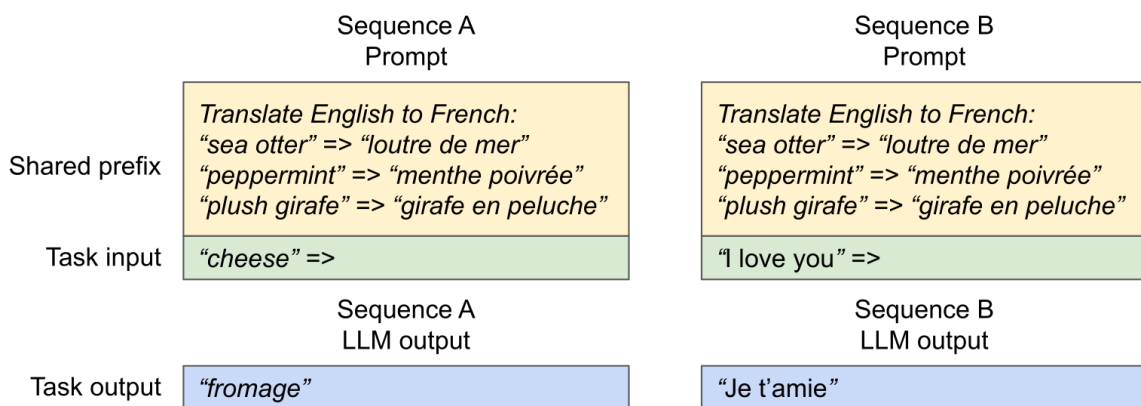
- b ：每块容纳的 token 数（vLLM 默认 16）；
- $\frac{b-1}{2}$ ：每条序列期望的内部碎片。

代入 $b = 16$ ：每序列期望浪费 7.5 个 token，对 13B MHA 约 $7.5 \times 0.82 \approx 6$ MB——与连续预留下动辄 GB 级的浪费相比降低两个数量级以上。块越大，碎片期望越大、block table 越小；块越小则相反，这是分页大小的经典权衡。

分页减少的是预留、碎片和复制浪费，不会凭空减少一条既定序列真正需要保存的 KV；attention kernel 仍要借助 block table 找齐相关数据。

8.6 前缀共享与 block-level copy-on-write

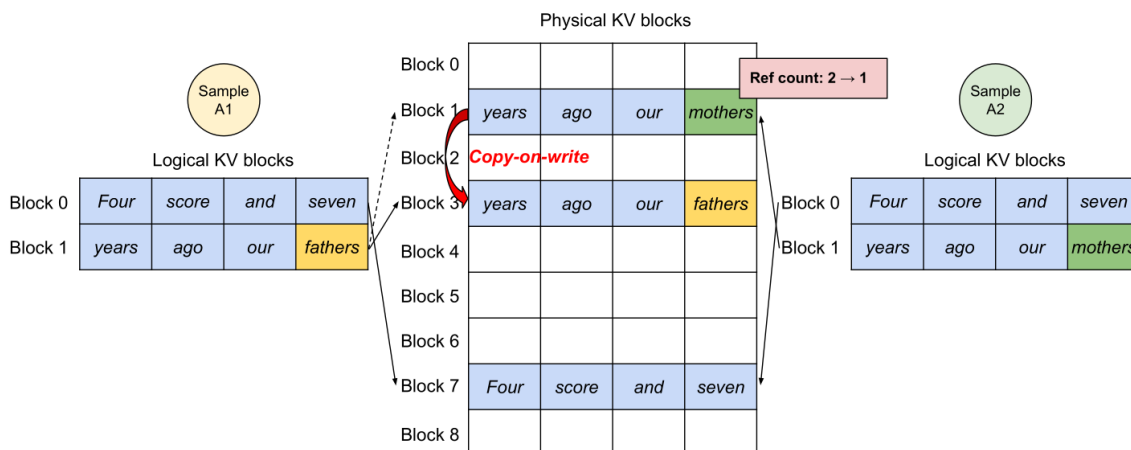
共享 system prompt、few-shot 示例，或同一 prompt 生成多个候选时，多条逻辑序列可以指向同一组 physical blocks。前缀完全相同且引擎维护 prefix cache 时，这既减少存储，也可以避免重复 prefill。



两条翻译请求共享 *instruction* 和三条示例，只在 *task input/output* 分叉。

引用计数机制与 OS 的共享页相同：每个 physical block 维护一个 refcount，记录有多少条逻辑序列的 block table 指向它。分叉以前只增加引用计数，不复制数据；当某条序列要在共享且尚未填满的块里写入不同 token 时，系统才复制该块，这就是 copy-on-write。完整的一次 CoW 写流程是：

1. 写入方定位目标 logical block，查到 physical block 的 refcount；
2. 若 refcount 为 1，独占，原地写入；
3. 若 refcount 大于 1，分配新物理块，把旧块内容拷入，旧块 refcount 减 1，新块 refcount 置 1，block table 改指新块，然后写入。



两个 *sample* 共享 *Four score and seven / years ago our*，最后分叉成 *fathers* 与 *mothers* 时复制块，引用计数从 2 降为 1。

这套机制的会计收益在“同 prompt 采 n 个样本”的场景下最直观：长度为 S 的共享前缀，KV 存储从 n 份降为 1 份加各自的私有后缀，prefill 也从 n 次降为 1 次——对 best-of- n 、beam search 和并行 Agent rollout 都是直接的成本除数。

vLLM 还结合了融合 block read 与 attention 的 kernel、FlashAttention/FlashDecoding 和 CUDA graphs。它们分别减少额外 HBM 流量或 kernel launch overhead，但视频因时间只快速点名，本文不扩写未讲实现。

核心提示 Continuous batching、selective batching 与 PagedAttention 不解决同一个问题：前者管理每轮有哪些请求，第二个决定不同算子如何处理 ragged shape，第三个管理 KV 的物理布局、共享和复制。

8.7 本章小结

- Continuous batching 在每个 decode step 弹出完成请求、加入新请求，避免静态 batch 空等。
- Selective batching 让 attention 尊重各序列长度，同时把非 attention token 拼成大矩阵运算。
- PagedAttention 用 logical-to-physical block mapping 消除连续预留要求，减少碎片。
- 引用计数、前缀共享与 copy-on-write 让同 prompt 多样本不必复制完整 KV。

9 总结与延伸

9.1 讲者的实质结论

这节课先证明同一个 Transformer 在训练和推理中呈现不同系统形态：训练可以形成规则的大矩阵运算；自回归 decode 却经常 memory-bound，而且请求的到达、长度和结束时间都动态变化。因此推理优化必须同时包含模型和系统两侧：

- 用 GQA、MLA、CLA、local/sparse attention 缩小请求私有状态；

- 用量化减少每个数的字节，用剪枝/蒸馏减少结构；
- 用投机采样把大模型的串行生成改成小模型猜测与大模型并行验证；
- 用 continuous batching、分页、前缀共享和 COW 处理动态工作负载。

回看全讲的公式链条，这四条路线作用的对象可以被精确定位：第一条缩小 $M_{KV/seq}$ ，第二条缩小 M_{param} 与位宽，第三条把 decode 的串行步数除以每轮期望前进 token 数，第四条把有效 batch 与显存利用率推向理论曲线。它们彼此正交，因此真实系统是组合使用——而组合的总收益可以直接代入 4.2 的带宽下界模型逐项估算。

讲者最后提出了比“把现有 kernel 再优化一点”更大的研究问题：标准 attention 的 KV cache 与自回归组织方式，使 Transformer 从根本上不够 inference-friendly。线性 attention、状态空间模型与扩散式生成等架构，可能从设计起点改变状态规模或生成并行性。

这是一项有条件的研究展望，不是“某条路线已经解决全部问题”。新架构仍要同时证明：训练稳定、模型质量不降、长上下文能力可靠，并且在真实硬件上取得端到端收益。

9.2 一套可迁移的推理优化决策树

面对一个真实推理系统，可以按以下顺序诊断：

1. **先分阶段**：TTFT 主要看排队和 prefill，稳态 token latency 看 decode；
2. **再判瓶颈**：用 FLOPs/byte 和实测 profiler 区分 compute、HBM、通信与调度；
3. **找共享对象**：权重能否被更多请求复用，KV 能否跨 heads/layers/prefixes 共享；
4. **减少表示**：是否能降低 head 数、latent 维、历史长度、数值位宽或模型结构；
5. **借回并行**：是否能用 continuous batch 或 speculative verification 扩大一次有效工作；
6. **守住语义**：有损方法重新测质量，无损采样核对分布，系统方法核对逻辑 attention 不变；
7. **做端到端验证**：局部字节数或 FLOPs 的下降，不等于产品 TTFT、latency 和 throughput 必然同步改善。

9.3 初学者应能独立完成的检查

读完本讲后，应当能够：

- 从 $B, S, T, D, F, N, K, H, L, V$ 写出参数与 KV cache 的一阶内存模型；
- 解释为什么 prefill 与 decode 的算术强度不同，为什么 batch 对 MLP 和 MHA attention 的作用不同；
- 比较 GQA、MLA、CLA、local/sparse attention 各自压缩的轴与可能损失；
- 手算一个标量量化例子，并解释 AWQ 为什么不是简单留下 1% FP16；
- 写出投机采样的接受概率与残差分布，用二元词表证明输出仍来自 target；
- 区分 continuous batching 的调度、selective batching 的算子组织和 PagedAttention 的内存管理；
- 用 logical block、physical block、引用计数和 copy-on-write 复述一次共享前缀分叉。

9.4 仍然开放的问题

- 长上下文中，怎样在精确检索、固定状态和 cache 成本之间取得更稳健的 Pareto 前沿？
- 投机采样的 draft 是否能动态适应任务、硬件和当前接受率，而不是固定选一个模型和 K ？
- Prefill 与 decode 是否应由不同设备、kernel 或调度队列承担？
- 新架构的“理论线性”优势，何时能转化为真实模型质量和端到端服务收益？

这些问题共同指向同一目标：不要只让模型“能生成”，而要让它以可持续、可测量、对硬件友好的方式生成。

9.5 本章小结

- 推理优化的共同目标，是在守住质量或目标分布的前提下，减少数据搬运、串行工作、状态容量和动态浪费。
- 模型架构、数值表示、采样算法与服务系统必须联合设计；单点优化很容易把瓶颈推到别处。
- Transformer 推理已经有大量成熟工程技巧，但推理友好的新架构仍有巨大、尚待验证的研究空间。